

Comparative Analysis of Sequential Models for Mobile Network Traffic Forecasting

Telecom Italia Milan call-detail records, November 2013 – January 2014

Hassan Adelani Luqman

Abstract

Mobile network operators must commit radio and backhaul resources before demand arrives, which makes short-horizon traffic forecasting an operational requirement rather than an academic exercise. This study compares three structurally different sequential models which includes a dynamic harmonic regression with ARIMA errors, a long short-term memory (LSTM) network, and a gradient-boosted tree ensemble (LightGBM), for one-step-ahead (10-minute) forecasting of mobile internet activity in Milan, evaluated on the week 16–22 December 2013 across three geographical areas with distinct traffic regimes.

19.38 GiB of raw call-detail records were processed into a 340.58 MiB matrix using a streaming pipeline whose working set is independent of dataset size. Exploratory analysis established that two seasonal cycles operate simultaneously, carrying 82.9% and 10.0% of variance, and that the lag-1 autocorrelation is 0.987, high enough that a naive persistence forecast is a demanding baseline and that any flexible model risks collapsing to reproducing its own input.

The dynamic harmonic regression achieved the lowest mean MASE (0.213), beating persistence by 11.3% and winning two of the three areas, using 22 parameters against LightGBM's 19,639–21,010 and the LSTM's 202,369. On the third area a three-seed LSTM ensemble was better. Averaged across areas only two of the three models beat persistence. On a held-out holiday period never used for tuning, persistence won outright on two of three areas while the two learned models degraded by 69–140%; the harmonic model stayed within 9%. An explicit diagnostic confirmed that no model had collapsed to persistence. Training and inference cost were found to invert: the smallest and most accurate model is roughly $1,825\times$ the most expensive to serve, and could not keep pace with the forecast interval at full-grid scale.

1. Introduction

Mobile network operators face a scheduling problem with a short deadline. Radio resources, backhaul capacity and base-station power must be committed before demand arrives, and the cost of guessing wrongly is asymmetric: under-provisioning degrades service for users, while over-provisioning wastes energy and capital. Accurate short-horizon forecasting is what makes proactive allocation possible, and it underpins practical techniques such as load balancing between cells and putting lightly loaded base stations to sleep during quiet periods [5].

This study investigates the problem empirically using telecommunications activity data collected across Milan by Telecom Italia between 1 November 2013 and 1 January 2014 [1], [11]. The city is divided into a 100×100 grid of 235 m square cells, and internet activity is recorded at 10-minute intervals, giving 8,928 observations for each of 10,000 areas over the two-month period. The dataset is public and widely used, which makes results on it comparable across studies in a way that proprietary operator data is not.

The research question is:

How do different sequential models compare for one-step-ahead mobile network traffic forecasting, and how does their performance vary across geographical areas with different traffic characteristics?

The question has two halves, and the second is easy to under-serve. Comparing models on a single series answers only which algorithm fits that series; it says nothing about whether the ranking is a property of the method or of the data it was given. This study therefore treats the choice of areas as a methodological decision rather than an administrative one. Section 4.2 shows that the obvious choice — the three highest-traffic cells — would have made the comparison uninformative, because those three cells turn out to be immediate neighbours behaving almost identically.

Three models are implemented and compared: a dynamic harmonic regression with ARIMA errors, an LSTM network, and a gradient-boosted tree ensemble (LightGBM). They are selected in Section 2 to differ in what they *assume* rather than only in how they are implemented — the first imposes seasonal structure explicitly, the second learns representations from a raw sequence, and the third learns from engineered features. Persistence and seasonal-naïve baselines appear in every results table as the reference floor.

The contributions are: an account of processing 19.38 GiB of raw call-detail records into a tractable form under stated computational constraints, with measured evidence for each decision; an exploratory characterisation of the data that establishes what a forecasting model must represent; a comparison of three structurally different models on areas chosen to span distinct traffic regimes; and an analysis of where those models fail, including a held-out period containing the Christmas and New Year holidays.

2. Related Work and Model Selection

2.1 The problem and this dataset

Wang *et al.* [5] survey cellular traffic prediction and identify resource allocation, load balancing and base-station sleep scheduling as the recurring motivations, alongside the observation that 5G deployment has made both data volumes and traffic patterns harder to model than in earlier generations.

The dataset used here occupies an unusual position. Barlacchi *et al.* [1] released it as part of the Telecom Italia Big Data Challenge, and because it is public, large and spatially resolved it has become a common benchmark. Two prior studies are directly relevant because they use the same Milan grid. Zhang and Patras [3] forecast network-wide traffic with a spatio-temporal neural network, treating the grid as an image sequence and combining convolutional and recurrent components, reporting up to 61% lower error than established baselines at horizons up to ten hours. Santos *et al.* [4] address the short-horizon problem this study addresses, comparing LSTM and GRU against Random Forest and Decision Tree on two months of Milan data, clustering cells by activity level before modelling.

Santos *et al.* is the closest published analogue, and their clustering step is worth noting: they too found that cells differ enough that treating them uniformly is unsatisfactory. This study reaches the same conclusion from a different direction, as Section 4.2 sets out.

2.2 Statistical approaches, and why the obvious one fails here

ARIMA and its seasonal extension SARIMA are the standard statistical baselines, and much of the cellular forecasting literature positions itself against them. Azari *et al.* [2] provide the most careful comparison, evaluating LSTM against ARIMA on real network traffic and finding LSTM superior in general but identifying conditions under which ARIMA performs close to optimal at considerably lower complexity. A study that discarded statistical models on the strength of the headline result alone would be ignoring that paper's actual conclusion — and, as Section 6 reports, the present results support the qualification rather than the headline.

Seasonal ARIMA is nevertheless not usable on this data, for a structural reason. At 10-minute resolution the daily period is $s = 144$, at which SARIMA's seasonal differencing and parameter estimation become computationally intractable. More fundamentally, SARIMA accommodates **one** seasonal period, and the decomposition in Section 4.4 finds two.

The standard remedy is **dynamic harmonic regression** — Fourier terms at the required seasonal frequencies entered as exogenous regressors, with ARIMA errors capturing what remains. Hyndman and Athanasopoulos [6] set this out as the recommended approach precisely for long seasonal periods and multiple seasonalities. The decomposition method itself, MSTL, comes from the same literature: Bandara *et al.* [7] introduce it as an

extension of STL to multiple seasonal patterns, motivated by exactly the high-sampling-rate data used here.

2.3 Deep learning, and what it does and does not buy

The survey by Wang *et al.* [5] documents the field's movement toward deep learning, and the direction of travel is not in dispute. Three qualifications matter for this study.

The spatial architectures solve a different problem. Zhang and Patras [3] achieve their gains by using the whole grid, because neighbouring cells are informative when users move between them. This study is univariate by design. That is a real limitation rather than a neutral choice, and Section 8 treats spatial extension as the most promising future direction, with [3] as the evidence that it helps.

Recurrent architecture choice is not settled. Santos *et al.* [4] find LSTM and GRU both effective on this dataset, and GRU is the cheaper of the two. Selecting LSTM is a defensible default rather than a finding, and is stated as such.

Data hunger is a recognised failure mode. The survey literature notes that LSTM approaches generalise poorly when data is sparse. This study trains on 5,472 observations per area — adequate but not abundant, which is why early stopping and a regularisation search are used rather than assuming a large model will behave.

2.4 Gradient-boosted trees as a serious competitor

The strongest evidence against assuming a neural model will win comes from outside the networking literature. The M5 forecasting competition, reviewed by Makridakis *et al.* [8], concluded that machine-learning methods outperformed classical statistical ones — and the methods that did so were overwhelmingly gradient-boosted trees, with LightGBM [9] the most-used model among winning entries. Gradient boosting on lag features is therefore not a weak third option included for variety. Santos *et al.* [4] included Random Forest and Decision Tree on this dataset but not gradient boosting, which leaves a genuine gap this study occupies.

2.5 Evaluation practice

Hyndman and Koehler [10] show that many commonly used accuracy measures are degenerate in situations that occur routinely, and propose the mean absolute scaled error as a general-purpose alternative. MASE is scale-free and interpretable: below 1 means the model beats a naive one-step forecast on the training data. That property is necessary here, because the three areas modelled differ by roughly fivefold in mean volume. MAPE needs a stated caveat rather than silent use, being undefined at zero and unstable near it; the three areas have minima of 47.3, 55.8 and 80.2, so no near-zero denominator arises, and this is checked rather than assumed.

2.6 The resulting line-up

Model	What it assumes	Why it is included
Dynamic harmonic regression	Seasonal form is given explicitly	The approach [6] recommends for multiple long seasonal periods; the only model here that states the structure rather than learning it
LSTM	Nothing about seasonal form	Learns from a raw window; the dominant architecture in the single-cell literature [4], [5]
LightGBM	Structure lives in engineered lags	Beat both alternatives at scale in M5 [8], [9]; the gap left by [4] on this dataset

The three differ in what they assume, which is what makes the comparison informative rather than a contest between implementations.

3. Dataset and Data Preparation

3.1 The data

The dataset comprises 62 tab-separated text files, one per day, totalling **19.38 GiB** (20,804,803,507 bytes), with individual files ranging from 265 to 359 MiB. Each record carries a square identifier, an interval start time in epoch milliseconds, a counterparty country code, and five activity measures, of which internet is the target variable.

Three properties of the raw format determined the whole processing design, and each was established by inspection rather than assumed.

Records are split by country code. A single (`square_id`, `time_ms`) pair appears once for every country that generated traffic in that cell and interval — on average 3.6 times, and at most 36. Aggregating over country is therefore not a tidying step but the operation that recovers the quantity of interest. Across the period, **319,896,289 raw records** reduce to 89,280,000 cells.

Timestamps are UTC, but files are aligned to local midnight. The first record of the 1 November file carries 1383260400000, which is 2013-10-31T23:00:00Z — midnight on 1 November in Europe/Rome, since Italy was on CET. Treating the timestamps as UTC days would have displaced every diurnal pattern by an hour. No daylight-saving transition falls inside the observation window (Italy changed clocks on 27 October 2013 and 30 March 2014), so every local day contains exactly 144 intervals; this was asserted during assembly rather than assumed.

The target is normalised activity, not an integer count. Values such as 11.028366381681026 appear throughout. The measure is a normalised activity index derived from call-detail records, and is described as such throughout this report.

3.2 Processing strategy

After aggregation the data is small: the full 8928×10000 matrix occupies **340.58 MiB** as float32. The memory-management problem is therefore not storing the result but reaching it, converting 19.38 GiB of text without ever holding more than one day in memory.

Only three of the eight columns are required. The pipeline projects to `square_id`, `time_ms` and `internet` at parse time, discarding five-eighths of the data before it is materialised, casts to the narrowest sufficient types (uint16, int64, float32), and aggregates over country code within a single pass. Each day is reduced to a dense 144×10000 block of 5.49 MiB and written to disk before the next is read.

Three strategies were implemented and measured rather than one being assumed best: a naive pandas read (every column, inferred dtypes, whole file resident) implemented to be measured rather than used; a chunked pandas path with projection, narrow dtypes and incremental aggregation; and a Polars [13] lazy scan with projection pushdown into the CSV reader and a streaming group-by.

3.3 Measured results

Table 1 reports each strategy on identical input. Peak resident set size is the figure of interest rather than the size of the resulting frame: the peak occurs during parsing and is released before the frame is returned, so it is sampled on a background thread at 20 ms intervals rather than read once at completion.

Table 1 — Ingest strategies, one day (2013-11-01), local hardware.

Strategy	Resident	Peak RSS	Wall time	Scales with days
Naive pandas	295.57 MiB	633.06 MiB	5.56 s	yes

Strategy	Resident	Peak RSS	Wall time	Scales with days
Chunked pandas	5.49 MiB	172.55 MiB	5.86 s	no
Polars lazy	5.49 MiB	552.22 MiB	0.95 s	no

The result contradicted the expectation that the faster engine would also be the leaner one. Polars is roughly six times faster per day but peaks **3.2× higher** than chunked pandas, because its CSV reader holds the entire ~322 MB file in memory before parsing. Two candidate remedies were measured and both made matters worse: `low_memory=True` reached 559 MiB, and batched reading 697–862 MiB depending on batch size. Neither strategy dominates, so both are retained and selected by a command-line flag; the full run used Polars, because the execution environment had ample memory and throughput was the binding constraint.

The figure that matters most is not peak memory but scaling. Holding all 62 days in the naive representation would require **17.90 GiB** resident — an extrapolation from one day, stated as such, and deliberately not executed. Both optimised paths hold a single 5.49 MiB block at any moment, so their working set is independent of the number of days processed. The pipeline is $O(1)$ in dataset size where the naive approach is $O(n)$, and that difference, rather than the ratio of peak values, is what makes the processing feasible.

A measurement artefact worth reporting

An early version of this benchmark ran all three strategies within one process and reported 544, 184 and 384 MiB for *identical* work. The cause is that CPython and the allocators beneath it do not return freed arenas to the operating system, so whichever strategy runs first absorbs the cost of growing the heap and later ones appear artificially cheap. Every measurement reported here therefore runs in a fresh subprocess; repeated runs under that protocol agree to within about 2%. Available system memory is recorded alongside each measurement, because memory pressure causes the operating system to trim working sets and compress pages, which *depresses* measured peak RSS and would understate demand.

3.4 Execution environment

The processing was performed on a cloud notebook platform rather than locally, for a measured reason. The download rate available locally was 0.44 MiB/s (3.7 Mbit/s), implying approximately 12.6 hours for the full dataset. The same transfer on the remote platform sustained **55.9 MiB/s (469 Mbit/s)** and completed in **5.92 minutes** — a factor of roughly 127. Since the raw data is required exactly once, to produce a 340.58 MiB artefact, performing that conversion where the bandwidth is available was substantially more efficient than performing it where the analysis was developed.

The environment also imposed constraints that shaped the pipeline. Session scratch storage is discarded when a session ends and sessions are capped at 12 hours, so a design that downloaded all 62 files before ingesting any would, on a timeout, lose every raw file and leave nothing behind. The pipeline therefore interleaves: each day is downloaded, converted, written to persistent storage, and its raw text deleted before the next begins. Peak disk usage stays near the size of one file rather than 19.38 GiB, and every completed day is a durable artefact that a resumed run skips.

Table 2 — Full ingest run (62 days).

Quantity	Value
Raw records parsed	319,896,289
Download	5.92 min total, 5.73 s/day $\pm$ 0.64
Ingest	1.54 min total, 1.49 s/day $\pm$ 0.10
Total wall time	7.7 min

Quantity	Value
Peak RSS	574 MiB max, 326 MiB mean
Output	340.58 MiB

3.5 Missing data

Two distinct phenomena are easily conflated, and the pipeline treats them separately.

An **absent cell** is a (square, time) pair with no record in the raw file. Because a record exists only where activity was logged, an absent pair means no activity occurred, not that data was lost; such cells are set to 0.0 and the count reported. Across the period there are **34,682** of them, 0.0388% of the 89,280,000 cells.

A **missing interval** is a timestamp absent from the grid entirely, which would be genuine data loss. The policy specified for this case was to interpolate gaps of up to three intervals, leave longer gaps as NaN with the count reported, and raise an error rather than impute if any gap fell inside the evaluation week. In the event, the assembled matrix contains **zero missing intervals and zero NaN values**: the timestamp grid is complete, every local day contains exactly 144 intervals, and the policy never executed.

The absent-cell counts are not uniform across the period. They hold at roughly 22 per day through 21 November, step upward around 23 November, and reach 1,463 per day over the Christmas period, peaking at **2,267 on 26 December** (Santo Stefano). By split the means are 220 per day in training, 864 in the test week, and 1,463 in the held-out stress period — roughly a fourfold difference in data density between training and evaluation. Raw record counts over the same window decline smoothly by 6.7% rather than stepping, which indicates quiet peripheral cells falling into complete silence as overall activity declines, rather than a failure of collection. The pattern tracks human activity, peaking over the holidays, which is itself evidence for the interpretation of absent cells as silence. It does not affect the forecasting experiments: the three areas modelled contain no zero value at any point in any split, with minima of 47.3, 55.8 and 80.2.

3.6 Reproducibility

All configuration — paths, dates, split boundaries, preprocessing parameters, resides in a single YAML file, loaded into immutable structures with strict validation that rejects unknown keys and verifies that the four splits are contiguous, non-overlapping and within the observation period. Logic resides in a tested library of 501 tests; notebooks import from it and render figures. The processed artefacts other than the full matrix total approximately 600 KB and are version-controlled, so the exploratory analysis, the modelling stages and every figure in this report reproduce from a clean checkout without repeating the 19.38 GiB download. Appendix A gives the procedure and its verification.

4. Exploratory Analysis

4.1 How traffic is distributed across the city

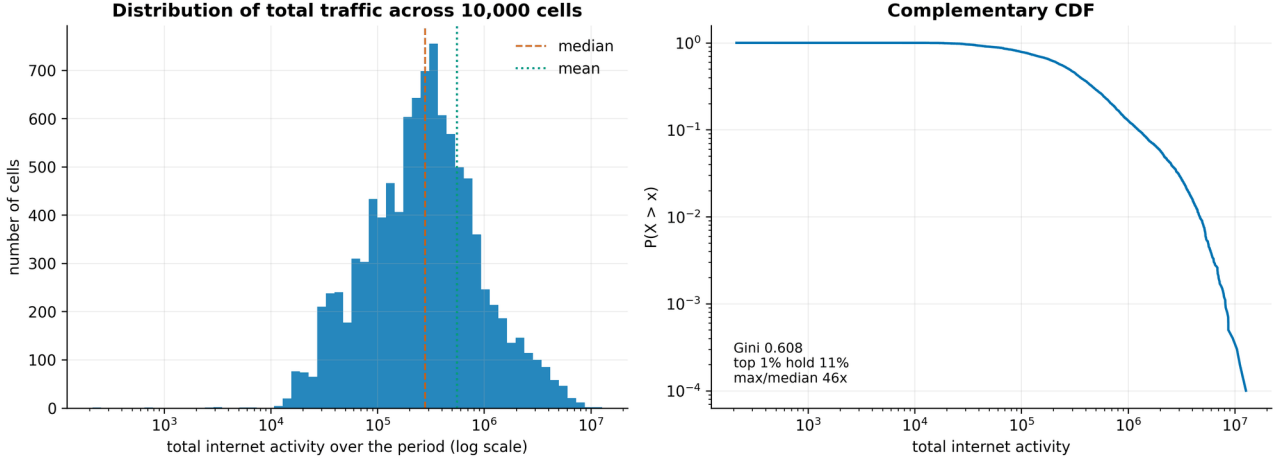


Figure 1 — Distribution of total internet activity across the 10,000 cells: histogram on a logarithmic axis (left) beside the complementary cumulative distribution function on logarithmic axes (right). Both use log scales because cell totals span nearly five orders of magnitude, from 213.6 to 12,740,060; on a linear axis almost the entire grid collapses into the first bin.

The distribution is strongly right-skewed (skewness 4.26, excess kurtosis 25.50) with a mean of 555,289 against a median of 277,871, and a maximum 45.8 times the median. Spatial concentration is substantial but not extreme: the Gini coefficient is **0.608**, the busiest 1% of cells carry **11.0%** of all traffic, the top 10% carry **48.4%**, and the least active half of the grid accounts for **11.6%**. No cell is entirely inactive over the period.

A two-parameter lognormal distribution fits the totals with $\mu = 12.496$ and $\sigma = 1.211$. A Kolmogorov–Smirnov test rejects the fit at $p = 4.3 \times 10^{-5}$, but that result requires qualification rather than plain reporting. The test statistic is 0.0232 against a critical value of 0.0136 at $n = 10,000$ and $\alpha = 0.05$ — only 1.7 times the threshold — and the logarithm of the totals has skewness +0.023 and excess kurtosis +0.013, both effectively zero. With ten thousand observations the test has sufficient power to reject a maximum cumulative discrepancy of 2.3 percentage points. The distribution is lognormal to any practical standard, and quoting the rejection without the statistic would misrepresent it.

The consequence for the study is that traffic volume varies by more than an order of magnitude between areas, so absolute error metrics such as MAE and RMSE are not comparable across them. The cross-area comparison in Section 6 therefore uses MASE, which is scale-free by construction [10].

4.2 Where the busiest cells are, and why it matters

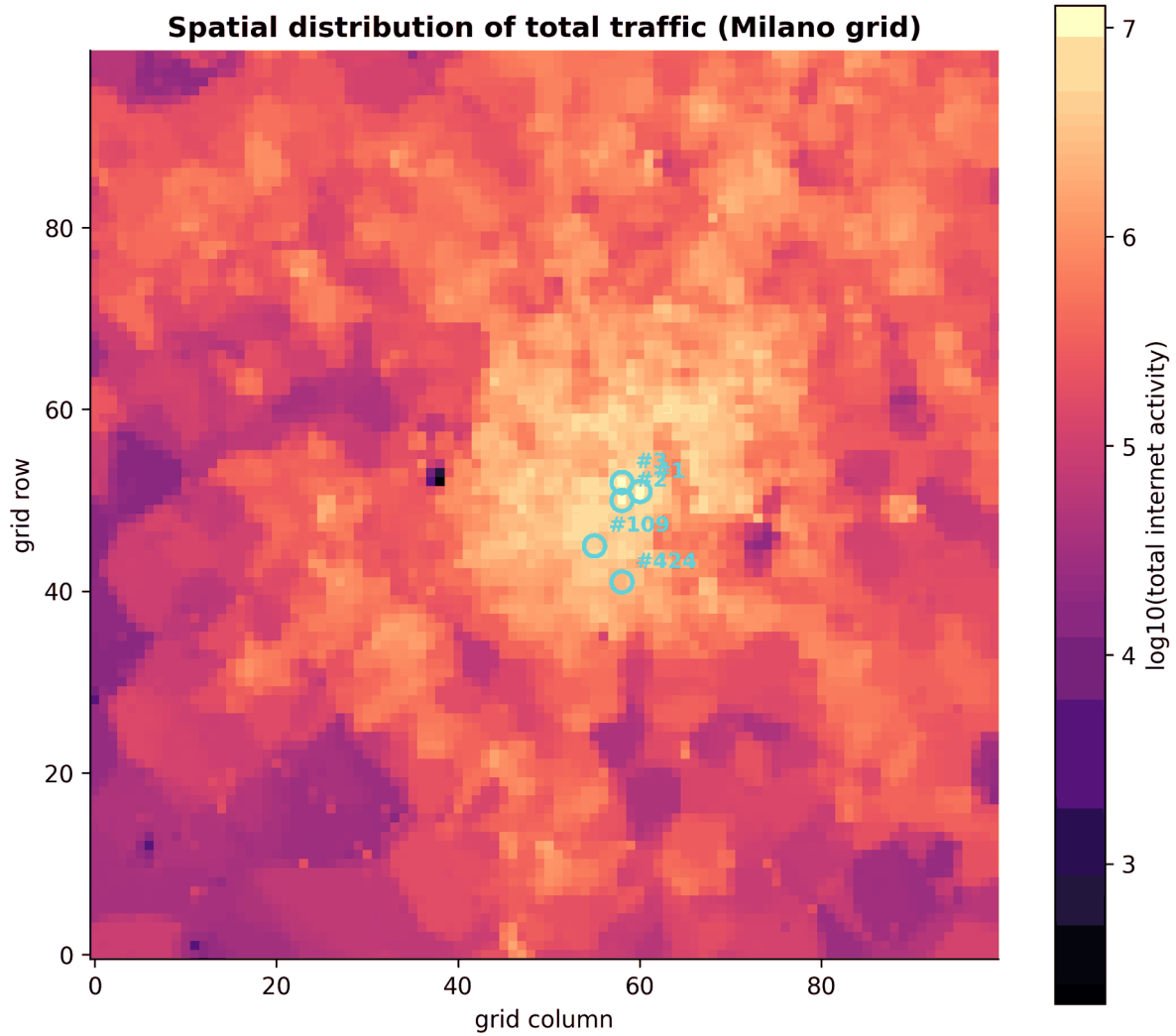


Figure 2 — Base-ten logarithm of total activity over the 100×100 grid, with the study areas marked. Activity is concentrated in a compact central region with a pronounced peak, decaying toward the periphery.

The three highest-traffic cells are squares **5161**, **5059** and **5259**, with totals of 12,740,060, 11,170,854 and 10,485,779. Converting their identifiers to grid coordinates places them at rows 50–52 and columns 58–60 — **within 470 m of one another**. The ten busiest cells occupy a bounding box of 13×10 cells, approximately 3.05×2.35 km, in central Milan.

This finding determined the study design. The brief specifies forecasting across three geographical areas, and the apparently natural choice is the three with the highest total traffic. But those three are immediate neighbours within a single hotspot, and a comparison across them would measure model performance on three samples of the same traffic regime. It could not answer how performance varies across areas with different characteristics, which is half of the research question.

The three areas modelled are therefore **square 5161** (rank 1), **square 4159** (rank 424) and **square 4556** (rank 109). Squares 5059 and 5259 are retained in the exploratory figures for completeness but are not forecast.

4.3 The five series and their characteristics

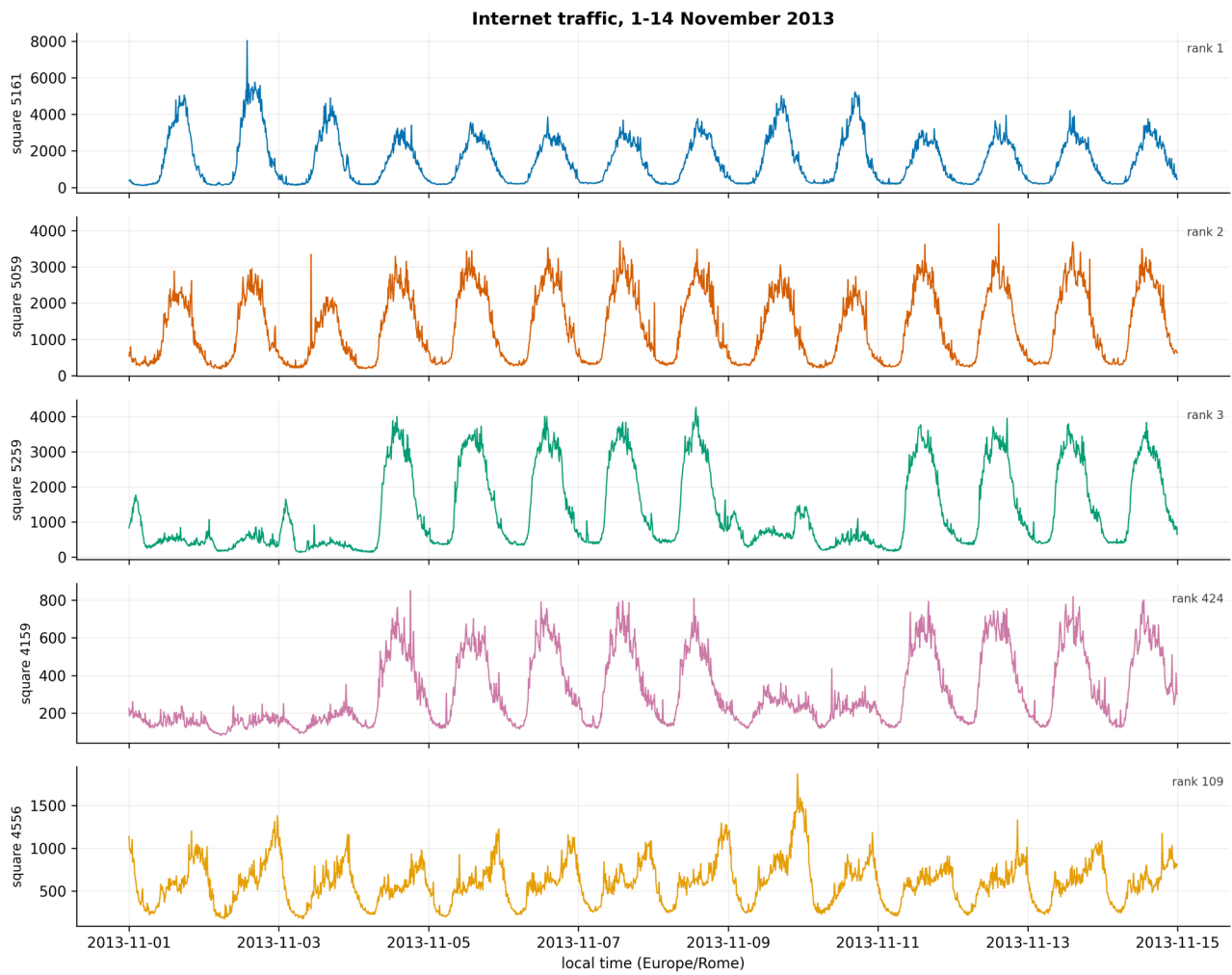


Figure 3 — Each of the five areas over the first two weeks of the observation period. Each panel keeps its own vertical scale, because the areas differ by an order of magnitude in volume and a shared scale would flatten the smaller ones into a line.

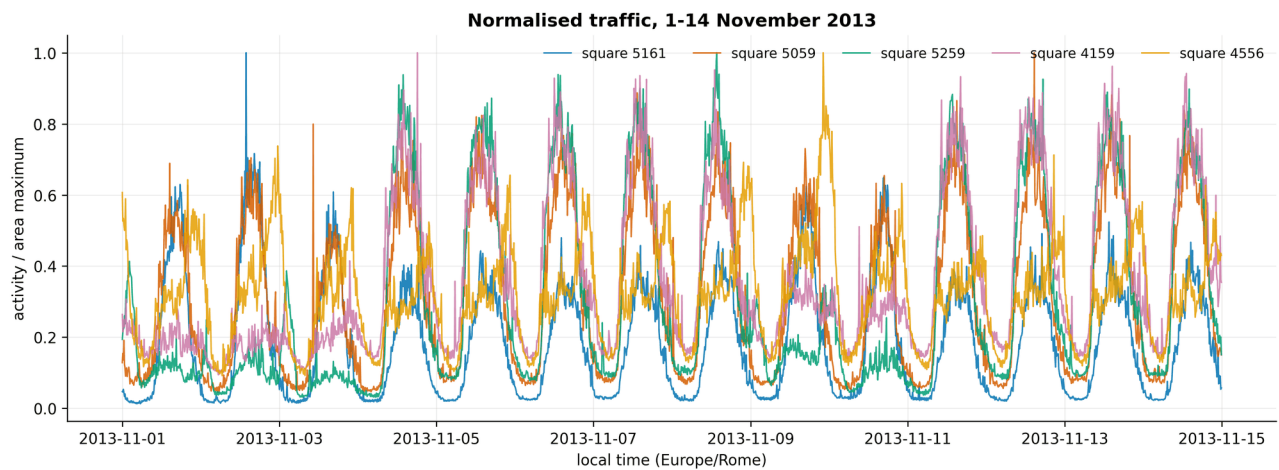


Figure 4 — The same five series normalised to their individual maxima, which compares shape rather than volume. All five show a pronounced daily cycle and a clear weekly modulation, with activity collapsing overnight and rising through the morning.

Table 3 — Per-area characteristics over the full period.

Square	Rank	Mean	SD	CV	Peak/trough	Night floor ÷ mean	Weekend ÷ weekday
5161	1	1,427	1,382	0.968	99.4	0.130	1.384

Square	Rank	Mean	SD	CV	Peak/trough	Night floor ÷ mean	Weekend ÷ weekday
5059	2	1,251	961	0.768	30.9	0.235	0.861
5259	3	1,174	1,103	0.939	47.0	0.333	0.425
4159	424	275	181	0.660	14.9	0.489	0.587
4556	109	512	248	0.485	17.0	0.534	1.140

The table justifies the area selection more strongly than the rankings do. The three forecast areas differ in the properties that make forecasting hard, not merely in volume. Square 5161 swings by a factor of 99 between its peak and its first-percentile trough and falls to 13% of its mean overnight; square 4556 swings by a factor of 17 and retains a night floor at 53% of its mean. Their coefficients of variation differ by a factor of two.

The weekly patterns separate them further, and the grid geometry published with the dataset [12] allows the pattern to be checked rather than merely inferred.

Table 4 — Location of each study area, from the published grid geometry.

Square	Centroid (lat, lon)	Nearest reference point	Distance	Weekend ÷ weekday
5161	45.4655, 9.1934	Galleria Vittorio Emanuele II	276 m	1.384
5059	45.4634, 9.1874	Duomo	226 m	0.861
5259	45.4676, 9.1874	Teatro alla Scala	167 m	0.425
4159	45.4443, 9.1873	Università Bocconi	365 m	0.587
4556	45.4528, 9.1783	Navigli	273 m	1.140

Two of the five match their temporal signature closely. Square 4556 lies 273 m from the Navigli, Milan's principal nightlife district, and has both the highest night floor of the five and a weekend ratio above one — the profile of an area that stays active late and is busier at weekends. Square 4159 lies 365 m from Università Bocconi, and its weekend ratio of 0.587 together with a high night floor is what a university quarter with substantial resident population would produce.

The three busiest cells are the more interesting result, and they qualify the reading above. All three sit within 500 m of the Duomo, and yet their weekend ratios span **0.425 to 1.384** — a factor of 3.3 between cells only a few hundred metres apart. Square 5259, closest to Teatro alla Scala and the offices around Piazza della Scala, empties at weekends; square 5161, beside the Galleria Vittorio Emanuele II, fills up. Proximity to a landmark is therefore evidence about a cell's location but not a sufficient explanation of its traffic.

That reinforces the case made in Section 4.2 from a second direction. It is not only that the three highest-traffic cells are spatially adjacent; it is that traffic character varies at a finer spatial scale than volume rank reveals, so ranking by total activity carries no information about temporal behaviour.

4.4 Multi-seasonal decomposition

The exploratory figures suggest two seasonal cycles operating simultaneously. Classical seasonal decomposition accommodates only one, so the series for square 5161 was decomposed using MSTL [7] with periods of 144 intervals (one day) and 1,008 (one week).

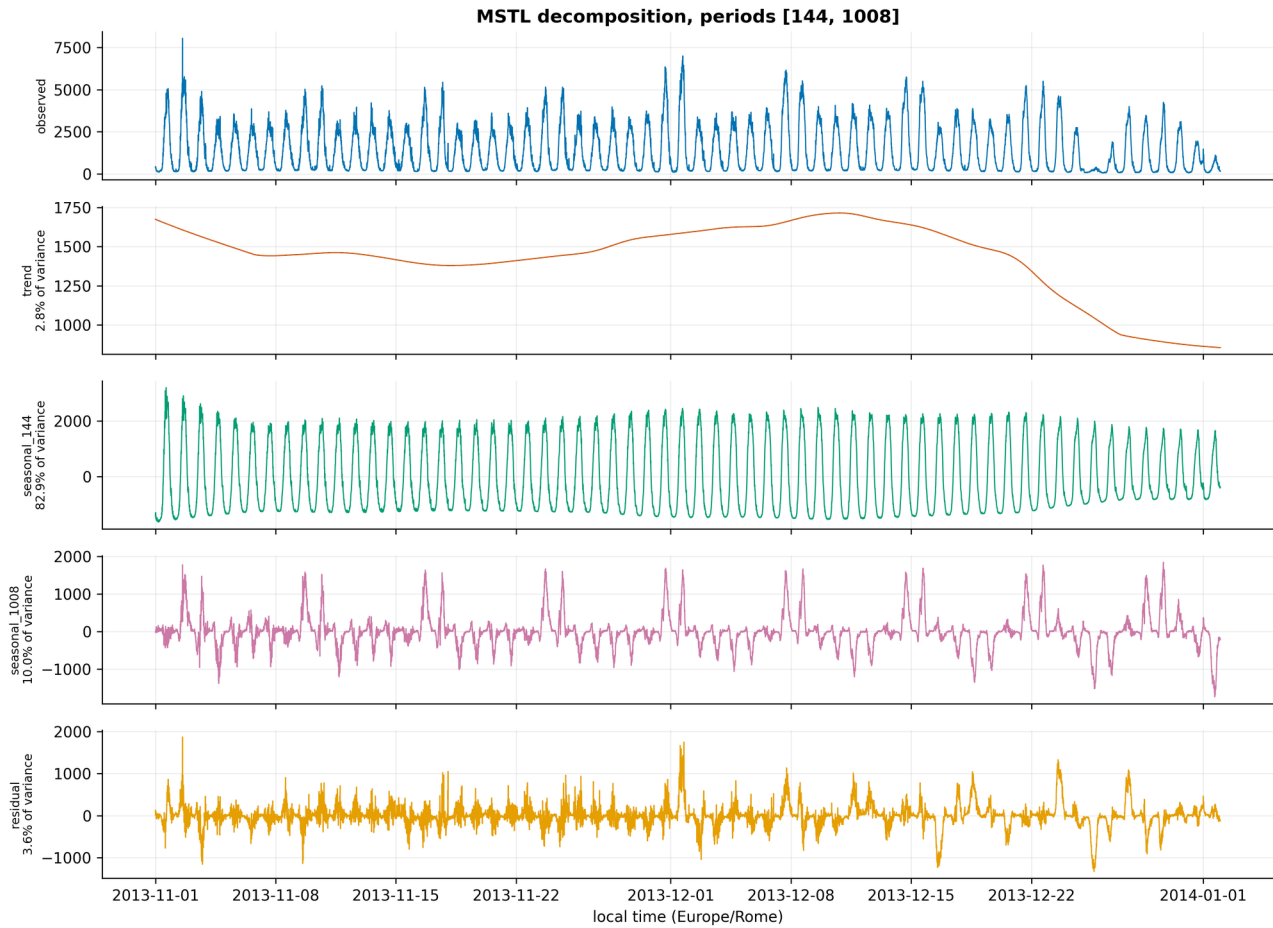


Figure 5 — MSTL decomposition of square 5161 into trend, daily seasonality, weekly seasonality and remainder.

Table 5 — MSTL variance decomposition, square 5161.

Component	Share of variance	Strength
Trend	2.8%	0.451
Daily seasonality (144)	82.9%	0.959
Weekly seasonality (1008)	10.0%	0.746
Residual	3.6%	—

Strength is measured as $1 - \text{Var}(\text{remainder}) / \text{Var}(\text{component} + \text{remainder})$, so a value near one indicates a component that dominates what remains after the others are removed. Variance shares do not sum to unity because the components are correlated.

Three conclusions follow. The daily cycle is overwhelmingly dominant, carrying 82.9% of variance at a strength of 0.959. The weekly cycle is secondary but not negligible at 10.0% and strength 0.746 — enough that a model ignoring it discards real structure. And the residual standard deviation is only **19% of the observed**, meaning roughly four-fifths of the variation is systematic structure that a model can in principle capture.

The modelling consequence is direct: two seasonal periods are present simultaneously, so the seasonal model must represent both. This is the argument for the dynamic harmonic regression introduced in Section 2.2 [6].

4.5 Autocorrelation and stationarity

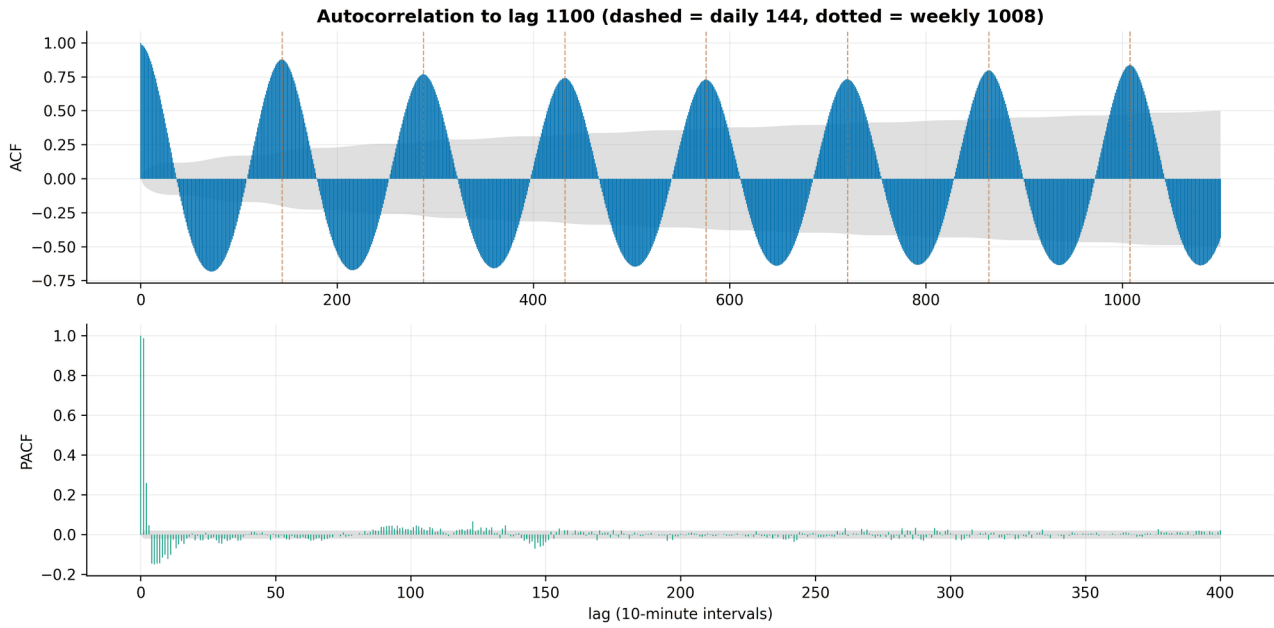


Figure 6 — Autocorrelation and partial autocorrelation for square 5161 to lag 1,100, with the daily and weekly lags marked.

The lag-1 autocorrelation is **0.987**. Local maxima occur at lags 144, 288, 432, 720, 864 and 1,008 — every multiple of the daily period, with the weekly lag among them — with autocorrelations of 0.878 at one day and 0.838 at one week. The structure decays slowly and periodically rather than dying out, which is the signature of strong seasonality rather than a short-memory process.

Stationarity was assessed with the Augmented Dickey–Fuller and KPSS tests together, since their null hypotheses are opposites and agreement between them is more informative than either alone.

Table 6 — Stationarity tests, square 5161.

Series	ADF statistic	ADF p	KPSS statistic	KPSS p	Verdict
Raw	−19.03	0.0000	0.237	≥ 0.10	Stationary (both agree)
First difference	−15.16	0.0000	0.003	≥ 0.10	Stationary (both agree)
Seasonal difference (lag 144)	−11.83	0.0000	0.069	≥ 0.10	Stationary (both agree)

KPSS p -values are clamped to the upper end of the tabulated range, so 0.10 indicates " ≥ 0.10 " rather than an exact value.

This result requires careful statement. Both tests indicate stationarity on the raw series, which would ordinarily suggest no differencing is required — and the harmonic regression is specified with $d = 0$ on this basis. But "stationary" here means only that the series contains no stochastic trend. The mean varies enormously with time of day, as Figure 3 and the decomposition both show; neither test examines that, because deterministic seasonality is not a unit root. Describing the series as stationary without this qualification would mislead.

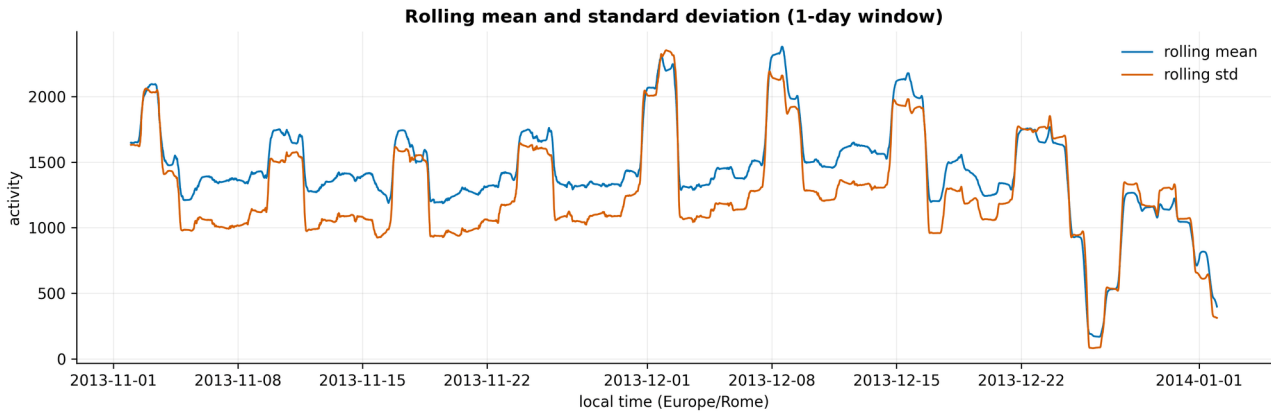


Figure 7 — Rolling mean and standard deviation over a one-day window, which makes the level stability visible while the within-day variation remains.

The very high lag-1 autocorrelation carries a second implication. A model can achieve a low one-step-ahead error simply by reproducing its most recent input, and would appear successful while having learned nothing. Persistence is therefore reported as a baseline in every results table, and Section 7.1 includes an explicit check for whether any model has collapsed to it.

4.6 Spectral analysis

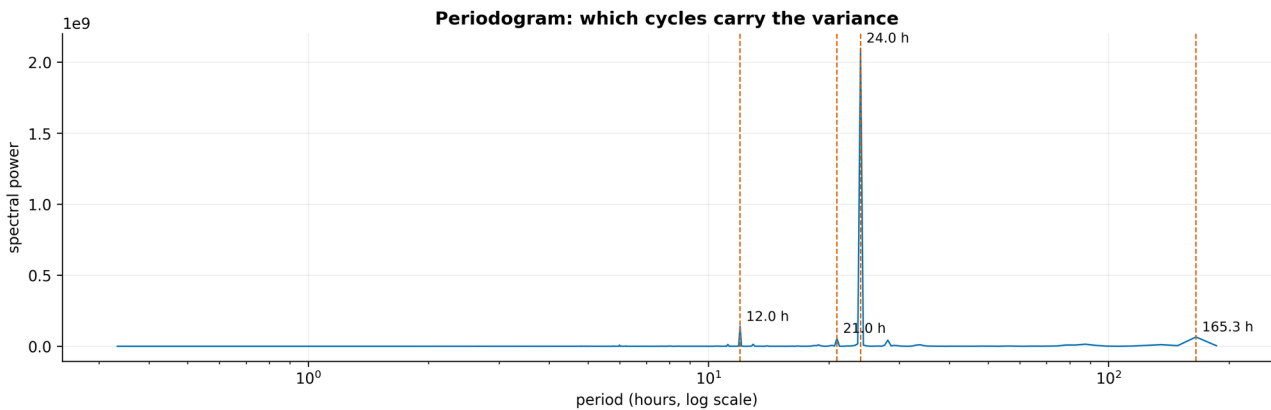


Figure 8 — Periodogram against period in hours, with dominant cycles annotated.

Power is concentrated at **24.00 hours**, as expected, with a substantial secondary peak at **12.00 hours** and a further peak near 165 hours corresponding to the weekly cycle.

The 12-hour peak is not an artefact. It indicates that the daily cycle is not a single sinusoid — it has structure within the day, consistent with distinct morning and evening activity. This has a direct consequence for model specification: a harmonic regression using a single Fourier pair for the daily period would capture the envelope of the daily cycle while missing its shape. At least two daily harmonics are required, and the harmonic order is selected empirically in Section 5.5 rather than assumed.

4.7 Anomalies and the holiday calendar

Intervals where a seasonal-naïve predictor fails unusually badly were identified, on the reasoning that periods difficult for the simplest seasonal model are likely to be difficult for all of them, and that identifying them in advance converts the later failure analysis from post-hoc explanation into a testable prediction.

The detection method required adjustment. Forecast errors on this series are strongly heteroscedastic: the standard deviation of the seasonal-naïve residual varies by a factor of **25 between the quietest and busiest**

hours of the day. A single robust scale estimated over the whole series is therefore determined by the quiet hours and flags every busy one — at a threshold of four robust standard deviations it marked 12.7% of all intervals, which identifies nothing. Estimating the scale separately for each position in the daily cycle asks the appropriate question, namely whether an observation is unusual *for that time of day*. A $\log_{1p}$ transform was evaluated as an alternative remedy and rejected: it reduces the spread ratio to 2.0 but still flags 2.85%, and since the study reports absolute-error metrics the absolute scale is the appropriate one on which to judge deviations.

Under the per-position scale, **152 intervals are flagged**, 1.73% of the 8,784 intervals that can be evaluated (the first day provides no seasonal-naive comparison). Holidays account for 12.9% of the days in the period but carry **23.0%** of the flagged intervals — an enrichment of **1.78×**, significant at $p = 4.3 \times 10^{-4}$ under a binomial test. Both Milan-specific dates appear: Sant'Ambrogio on 7 December, the city's patron saint's day and a municipal holiday coinciding with the opening of the La Scala season, and the Immacolata on 8 December. A national holiday calendar would have missed the former.

This justifies including a holiday indicator among the calendar features, since a model given only day-of-week has no means of anticipating a Wednesday that behaves like a Sunday. It also motivates the choice of held-out stress period: 23 December to 1 January contains four of the eight holidays in the observation window, and is reserved for failure analysis.

One case resists explanation. The single worst day is **Monday 2 December**, with 17 flagged intervals — more than Christmas Day (9) or New Year's Day (13) — and it is not a holiday. No cause has been identified. It is carried into the failure analysis in Section 7 rather than omitted.

4.8 Implications for the forecasting approach

The exploratory analysis establishes six constraints that the modelling approach must satisfy, each traceable to a measurement.

- **Two seasonal periods operate simultaneously** (82.9% and 10.0% of variance), so the seasonal model must represent both. Seasonal ARIMA cannot, and at $s = 144$ is in any case intractable.
- **The daily cycle is not sinusoidal** — the 12-hour spectral peak requires at least two Fourier harmonics for the daily period.
- **No differencing is required for a stochastic trend** (ADF and KPSS agree across all three transformations), fixing $d = 0$.
- **Lag-1 autocorrelation of 0.987 makes persistence a demanding baseline**, and creates a risk that a flexible model collapses to reproducing its input. Both a baseline and an explicit check are required.
- **The autocorrelation peaks at 144, 288, 432, 720, 864 and 1,008** identify the lag set for a feature-based model empirically rather than by convention.
- **Variance scales with level**. The correlation between daily mean and daily standard deviation is +0.947 on the raw scale and +0.265 after a $\log_{1p}$ transform, which justifies the variance-stabilising transform applied before model fitting.

5. Methodology

5.1 Problem definition and evaluation protocol

The task is one-step-ahead forecasting: given observations up to interval $t-1$, predict the internet activity at interval t , ten minutes later. Models are univariate and fitted per area, at the native 10-minute resolution.

Every reported forecast is produced by walk-forward inference with **true observed history**. The model predicts t from real observations up to $t-1$, then the real observation at t is revealed before $t+1$ is predicted. This is not a recursive rollout, in which a model would be fed its own predictions; that would measure multi-step error under a one-step-ahead heading. A single walk-forward implementation is used for all reported results, and the batched implementations used by LightGBM and the LSTM are asserted equal to the step-by-step loop in the test suite.

The period is split chronologically, never randomly, because random splits on a time series place future observations in the training set.

Table 7 — Chronological splits.

Split	Dates (Europe/Rome)	Points	Purpose
train	1 Nov – 8 Dec 2013	5,472	Model fitting; every transform fitted here only
validation	9 – 15 Dec 2013	1,008	Hyperparameter selection
test	16 – 22 Dec 2013	1,008	Reported results
stress	23 Dec 2013 – 1 Jan 2014	1,440	Failure analysis only; never tuned on

Transforms are fitted on the training split alone. The scaler raises an error if asked to refit, so leakage fails loudly rather than silently degrading into an optimistic result.

5.2 Metrics

MAE, RMSE, MAPE, sMAPE, R^2 and MASE are reported. MASE is the metric used for cross-area comparison, following Hyndman and Koehler [10], because the three areas differ by roughly an order of magnitude in volume: an MAE of 83.6 on square 5161 and 13.6 on square 4159 say nothing about which was forecast better. MASE scales each error by the in-sample naive error of that area's own training split, so a value below 1 means the model beats a naive forecast and values are comparable across areas.

The MASE denominator is computed on the **training** series in every case, including when scoring the validation, test and stress splits. Scaling by the window being scored would let an easy week flatter a model.

5.3 Baselines

Two baselines appear in every results table.

Persistence predicts $\hat{x}(t) = x(t-1)$. This is not a straw man. Section 4.5 measured a lag-1 autocorrelation of 0.987, and persistence achieves MASE 0.195 to 0.267 across the three test-week areas with R^2 above 0.94. It is the bar that decides whether any model's complexity was worth paying for.

Seasonal naive predicts $\hat{x}(t) = x(t-144)$, the same interval one day earlier. It tests whether the daily cycle alone is sufficient.

5.4 The three models

Dynamic harmonic regression. Fourier terms for the daily (144) and weekly (1,008) periods enter a SARIMAX model [14] as exogenous regressors while an ARIMA process models the error. The periodogram found a 12-hour component alongside the 24-hour one, requiring at least two daily harmonics, so the search began at $K^*_I = 2$. Both stationarity tests agreed the series has no unit root, so $*d$ was held at 0 throughout; differencing would have removed signal.

LSTM. Implemented in PyTorch [15]: a stacked LSTM over a window of recent observations with dropout and a linear head. It is the only model in the comparison that assumes nothing about seasonal form — the harmonic

regression is told the cycle explicitly and LightGBM is told which lags to examine, while the LSTM sees a raw window and must find whatever structure is there. It is therefore the only candidate for the behaviour the decomposition could not remove: a residual that is still 3.6% of variance and strongly heteroscedastic.

LightGBM [9]. Gradient-boosted trees over causal lag, rolling and calendar features. Its lag set was chosen from the measured autocorrelation rather than convention: lags 1, 2, 3, 6, 12, 144, 145, 288 and 1,008, being the strongest single predictor together with every multiple of the daily period that the ACF identified. Feature importances then provide an interpretability check that neither other model offers.

5.5 Input representation, preprocessing and training

The three models consume the same series in three different forms, and the differences are the substance of the comparison rather than implementation detail.

Variance stabilisation and normalisation. Section 4.8 measured a correlation of +0.947 between daily mean and daily standard deviation, falling to +0.265 after a $\log_{1p}$ transform: error variance scales with level, which violates the constant-variance assumption behind squared-error training. All three models therefore fit on $\log_{1p}$ -transformed values, and the LSTM and LightGBM additionally standardise to zero mean and unit variance. The statistics are those of $\log_{1p}(x)$, not of x — standardising first and taking logs afterwards would be a different transform and would not invert the same way. Mean and standard deviation are estimated on the **training split only**; the scaler refuses to refit and raises an error if asked, so leakage fails loudly. Every reported metric is computed after inverse-transforming back to original units.

Calendar features, used by the LSTM and LightGBM, are six columns: sine and cosine of minute-of-day, sine and cosine of day-of-week, a weekend indicator, and a holiday indicator covering the Italian national calendar plus the two Milan-specific dates identified in Section 4.7. The cyclical encodings are used rather than raw integers so that 23:50 and 00:00 are adjacent rather than maximally distant.

Table 8 — Input representation by model.

Model	Input at time t	Shape	Target
harmonic ARIMA	16 Fourier terms ($2 \cdot K_1$ daily + $2 \cdot K_2$ weekly) as exogenous regressors; ARIMA(3,0,1) state	(16,)	$\log_{1p}(x(t))$
LSTM	144 previous intervals $\times$ (1 value + 6 calendar)	(144, 7)	$\log_{1p}(x(t))$, standardised
LightGBM	9 lags (1, 2, 3, 6, 12, 144, 145, 288, 1008), 3 rolling means and standard deviations (windows 6, 36, 144), 6 calendar	(27,)	$\log_{1p}(x(t))$, standardised

Every feature is **causal**: each is computed from observations strictly before t . The lag set is the one Section 4.5 identified from the measured autocorrelation, not a convention — lag 1 as the strongest single predictor, then every multiple of the daily period the ACF marked, with lag 145 included because it is the daily lag of the previous interval.

Training procedure. The LSTM is trained with Adam on an L1 objective, matching the selection metric, with gradient-norm clipping at 1.0 and dropout between layers. Training stops on validation MAE with patience, and the **best checkpoint is restored rather than the last**, because the final epoch is usually not the best one. LightGBM likewise optimises an L1 objective with early stopping on the validation split. The harmonic regression is fitted by maximum likelihood and then updated through the evaluation window with `append(..., refit=False)`, which conditions on each new observation without re-estimating parameters — the operation that makes its walk-forward inference tractable, and, as Section 6.4 shows, also the reason its per-step cost is high.

5.6 Tuning protocol

Tuning ran on the highest-traffic area only (square 5161), selecting on validation MAE. Search proceeded one axis at a time, carrying the winner forward, so that each decision is attributable: a single joint search would produce a configuration with nothing to say about *why* any setting was chosen beyond that an optimiser preferred it.

Three deviations from the staged pattern, each for a reason:

- **Harmonic orders were selected by AICc**, not validation error, because refitting and walk-forwarding 15 candidates costs far more than an information criterion computed in-sample, and AICc is the standard criterion for this choice.
- **LightGBM used Optuna** over 30 trials, because `num_leaves`, `min_child_samples` and the sampling fractions trade off against one another; a staged sweep would fix each at a value chosen while the others were wrong. The search converged by trial 10; the remaining 20 trials improved validation MAE by 0.6%.
- **The LSTM used five staged sweeps** over sequence length, capacity, optimisation, regularisation, and a calendar-feature ablation. That ablation measured calendar features as worth **14.7%** of validation MAE (98.22 with, 112.67 without).

Every candidate — **101 in total** — appends a row to an append-only experiment log carrying its hyperparameters, validation metrics, wall time, parameter count, the source commit, and a non-empty rationale describing what the comparison implied for the next step.

Table 9 — Selected hyperparameters.

Model	Selection
harmonic ARIMA	$*K_1^* = 6$, $*K_2^* = 2$ Fourier orders; ARIMA(3, 0, 1)
LightGBM	68 leaves, learning rate 0.034, 320 trees after early stopping
LSTM	window 144, 128 hidden units $\times$ 2 layers, batch 32, dropout 0.0, weight decay 1×10^{-4}

Two observations on the harmonic selection are worth recording. AICc chose six daily harmonics, more than the minimum of two the periodogram required, indicating that the daily shape has structure down to roughly four-hour components. And the entire gain from the ARIMA order came from the moving-average term: (2,0,0) $\rightarrow$ (2,0,1) reduced validation MAE by 3.9%, while the three best orders spanned 0.08 MAE — a difference within noise, so ARIMA(3,0,1) was selected on a margin that does not distinguish it from the more parsimonious ARIMA(1,0,1).

5.7 Final fits

Once hyperparameters are selected the validation split has done its job, and withholding it would discard a week of data the model is entitled to learn from. The final fit therefore uses train + validation, and the test week is untouched until prediction.

This creates a problem for the two models that stop early, because no held-out set remains. Refitting with early stopping against the test week would be leakage; refitting with no stopping rule would overfit. The resolution is to carry the **complexity** found during tuning rather than the stopping rule: the LSTM trains for the epoch count that was best on validation with patience disabled, and LightGBM uses the 320 trees early stopping chose. Both numbers were fixed before the final fit and without seeing test data.

The LSTM is run over **three seeds** and reported as mean $\pm$ standard deviation, because a single seed reports one draw from a distribution and the spread here proves comparable to the differences between models.

5.8 Two quantities that are easy to confuse

The results tables report the LSTM twice, and the distinction matters.

- `lstm` is the mean of the three seeds' errors: what one typical trained model does.

- `lstm_ensemble` is the error of the three seeds' *averaged forecast*, which is a different and better forecast, since averaging predictions cannot increase absolute error.

The saved prediction series — and therefore every figure below — contains the averaged forecast, so both rows are reported, and the ensemble's cost is stated as three fits of training, three passes of inference and three times the parameters.

6. Results

6.1 Accuracy on the test week

Table 10 — MASE by area, test week (16–22 December 2013). Lower is better; 1.0 is the in-sample naive forecast.

Model	5161	4159	4556	mean	worst
harmonic ARIMA	0.241	0.166	0.230	0.213	0.241
LightGBM	0.249	0.186	0.267	0.234	0.267
persistence	0.267	0.195	0.257	0.240	0.267
LSTM (3-seed ensemble)	0.233	0.253	0.249	0.245	0.253
LSTM (mean of 3 seeds)	0.265	0.259	0.257	0.260	0.265
seasonal naive	0.975	0.626	0.680	0.760	0.975

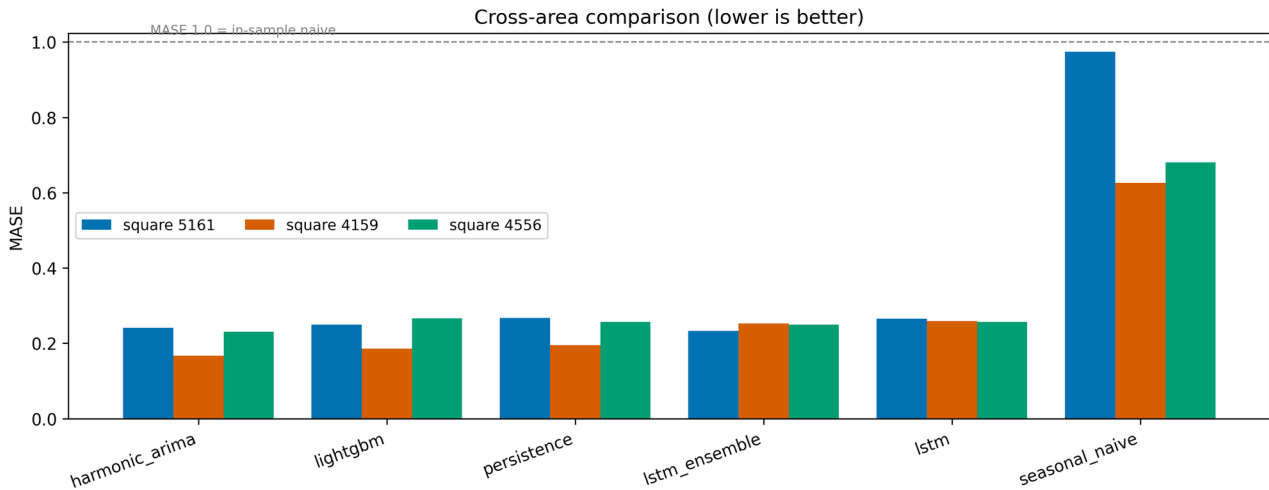


Figure 9 — MASE per model grouped by area, test week. The dashed line marks MASE 1.0, the in-sample naive forecast.

The dynamic harmonic regression has the best mean MASE and the best result on two of the three areas, improving on persistence by **11.3%** averaged across areas. On square 5161 the three-seed LSTM ensemble is better still, at 0.233 against the harmonic model's 0.241.

Averaged across areas, **only two of the three models beat persistence**: harmonic ARIMA and LightGBM. Both LSTM variants have a worse mean MASE than repeating the last observation. Seasonal naive is far worse than every alternative, which establishes that the daily cycle alone is not a sufficient forecast at this resolution.

The harmonic model achieves this with **22 parameters**, against 19,639–21,010 for LightGBM and 202,369 for a single LSTM. Ranking by parameter count inverts the ranking by accuracy. (LightGBM's count is the total number of leaves in its ensemble, the closest honest analogue to a parameter count for a tree model; unlike the other two it varies by area, because it depends on the data fitted rather than on the architecture.)

Table 11 — Full metrics by area, test week.

Area	Model	MAE	$\pm$	RMSE	MAPE %	sMAPE %	MASE	R ²
5161	LSTM ensemble	80.79	—	124.49	7.61	7.57	0.233	0.9917
5161	harmonic ARIMA	83.64	—	128.25	7.80	7.67	0.241	0.9911
5161	LightGBM	86.59	—	130.89	7.97	7.81	0.249	0.9908
5161	LSTM	92.01	3.03	142.67	8.48	8.44	0.265	0.9890
5161	persistence	92.80	—	134.88	9.19	9.10	0.267	0.9902
5161	seasonal naive	338.59	—	619.04	25.94	22.83	0.975	0.7934
4159	harmonic ARIMA	13.62	—	18.64	6.06	5.98	0.166	0.9766
4159	LightGBM	15.21	—	21.00	6.55	6.38	0.186	0.9703
4159	persistence	15.95	—	21.54	6.98	6.94	0.195	0.9688
4159	LSTM ensemble	20.67	—	29.74	7.88	7.56	0.253	0.9405
4159	LSTM	21.16	4.59	30.25	8.11	7.78	0.259	0.9364
4159	seasonal naive	51.19	—	84.64	21.80	20.49	0.626	0.5179
4556	harmonic ARIMA	25.87	—	34.70	5.93	5.82	0.230	0.9545
4556	LSTM ensemble	27.96	—	37.19	6.42	6.23	0.249	0.9478
4556	LSTM	28.85	1.79	37.96	6.71	6.51	0.257	0.9455
4556	persistence	28.86	—	39.62	6.60	6.55	0.257	0.9407
4556	LightGBM	29.93	—	39.66	6.98	6.76	0.267	0.9406
4556	seasonal naive	76.34	—	108.35	17.46	15.87	0.680	0.5567

6.2 Forecasts against observations

Figures 10–18 show each model against the observed series over the test week for each area. Every figure has the full week above and the busiest single day enlarged beneath, with persistence overlaid on the lower panel. The zoom is not decoration: at full-week width a thousand points are drawn across a few hundred pixels and every model traces the observed line, so a one-step lag only becomes visible at the lower scale.

Square 5161 (rank 1, highest traffic).

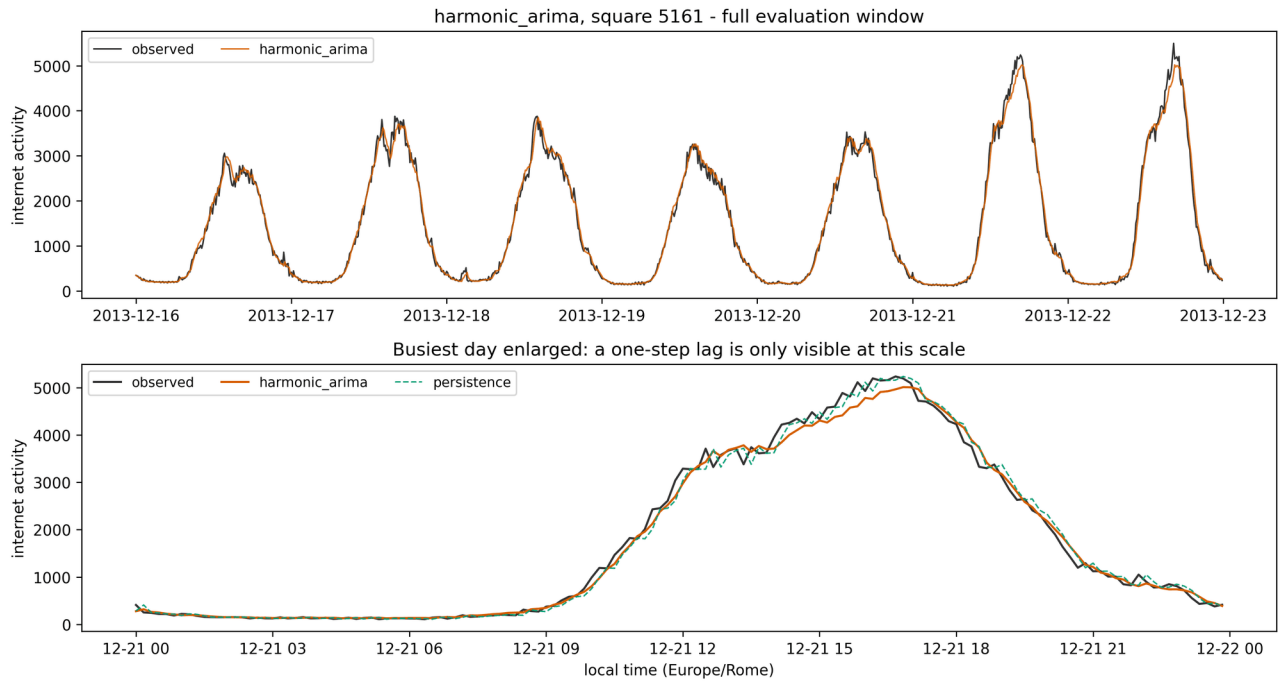


Figure 10 — Harmonic ARIMA, square 5161.

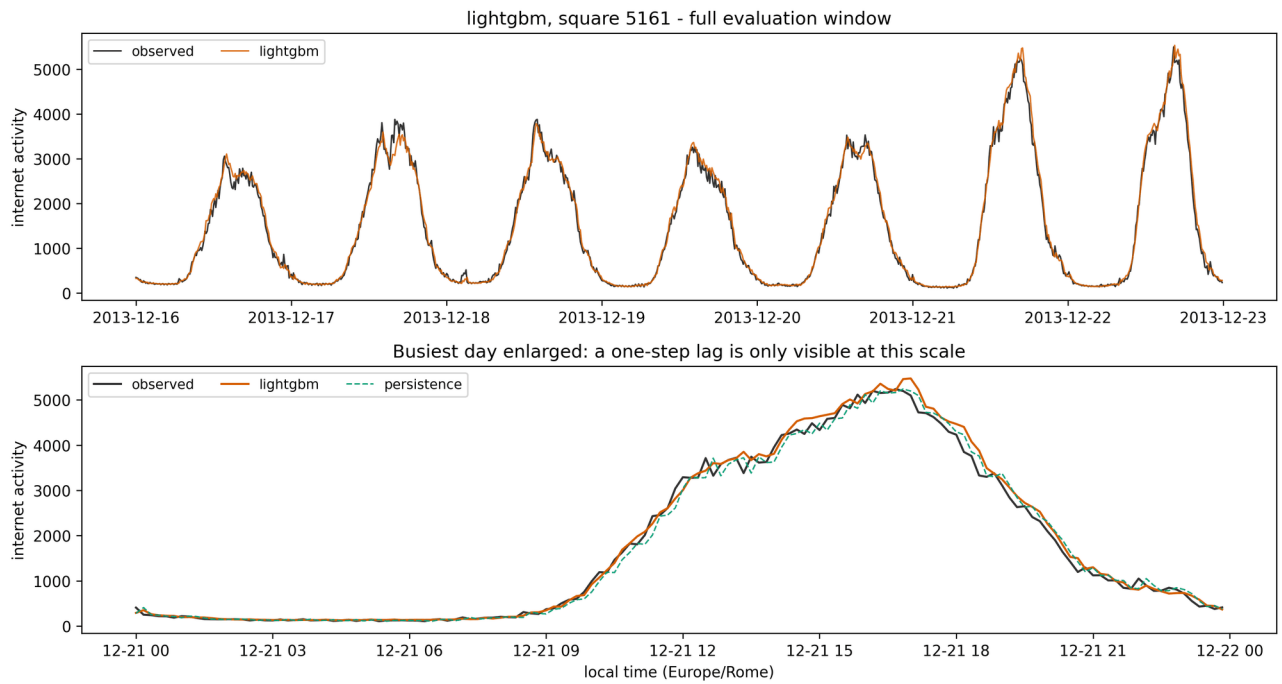


Figure 11 — LightGBM, square 5161.

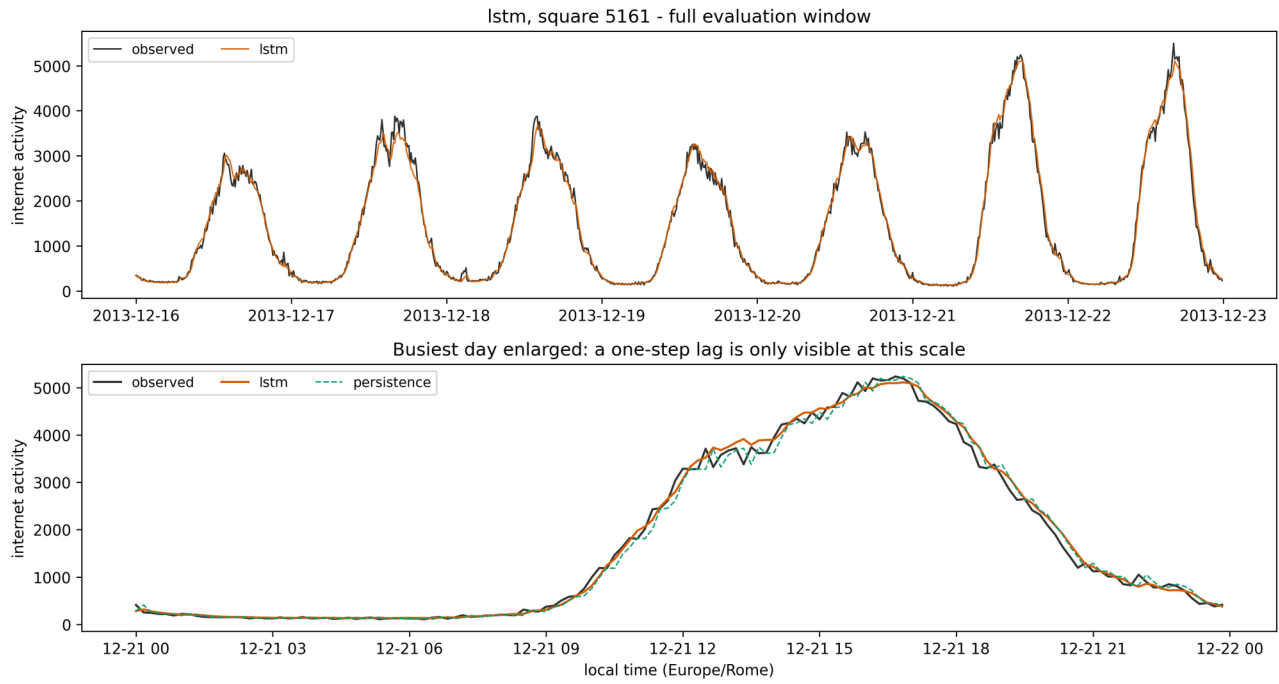


Figure 12 — LSTM (3-seed mean forecast), square 5161.

Square 4159 (rank 424, lowest traffic of the three).

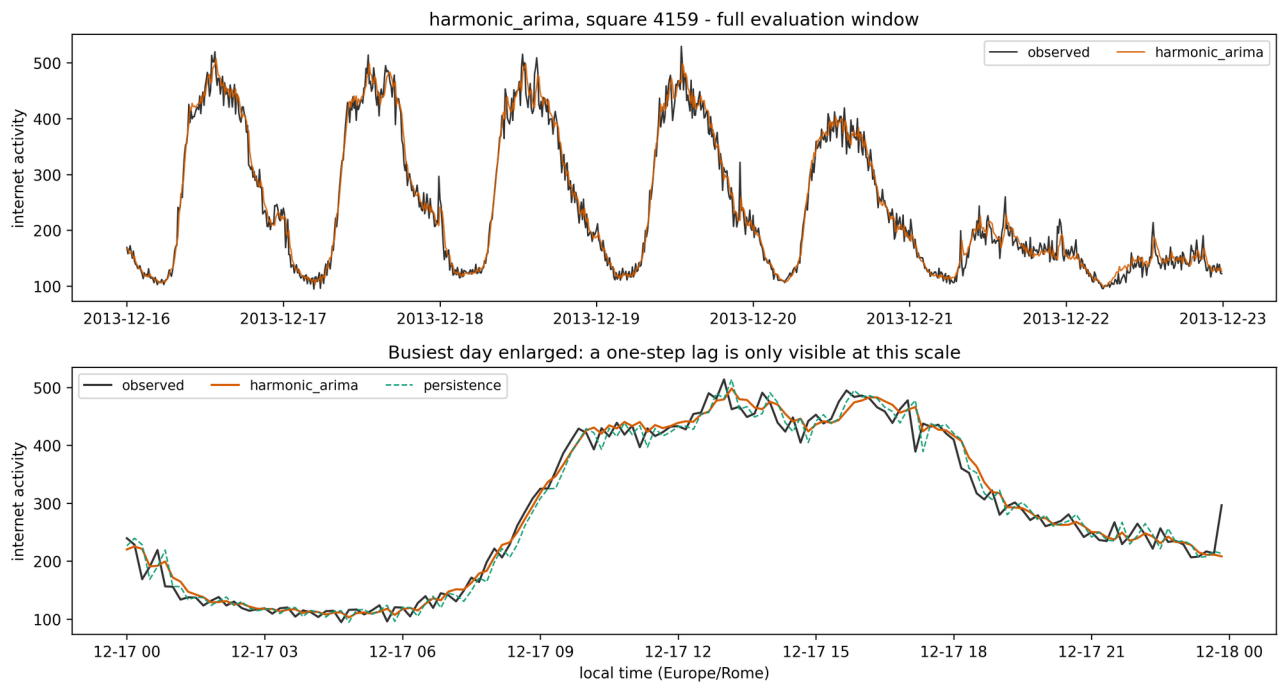


Figure 13 — Harmonic ARIMA, square 4159.

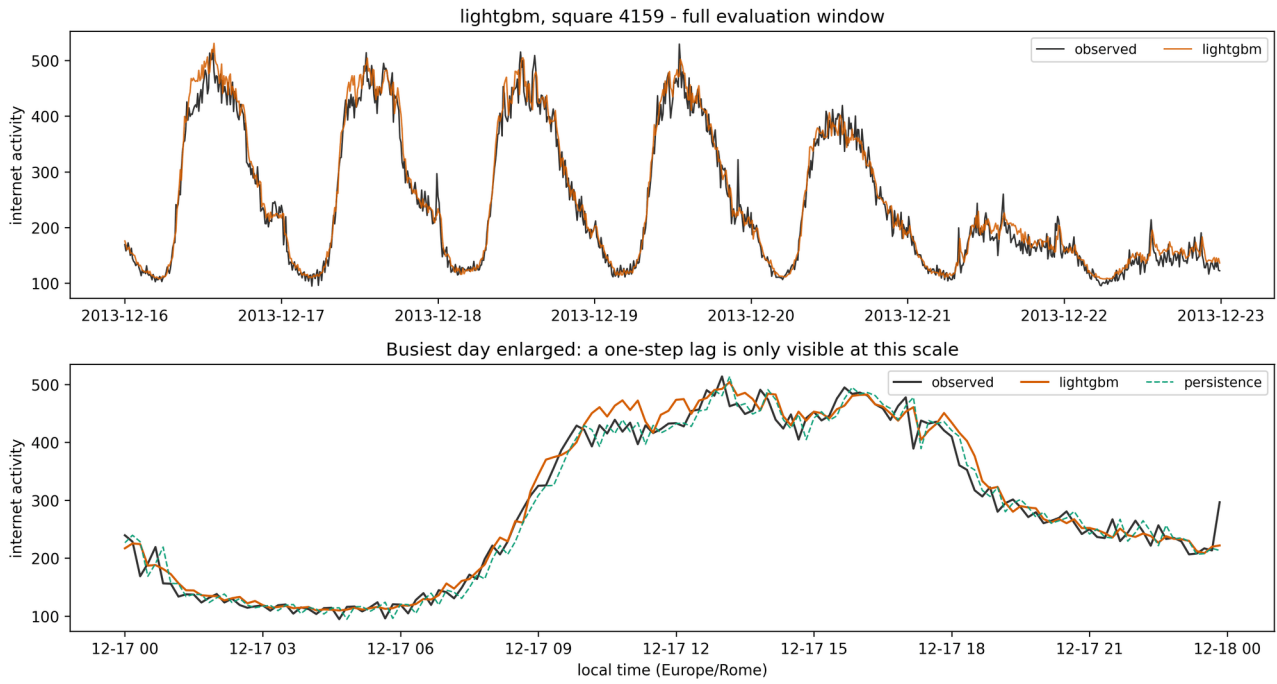


Figure 14 — LightGBM, square 4159.

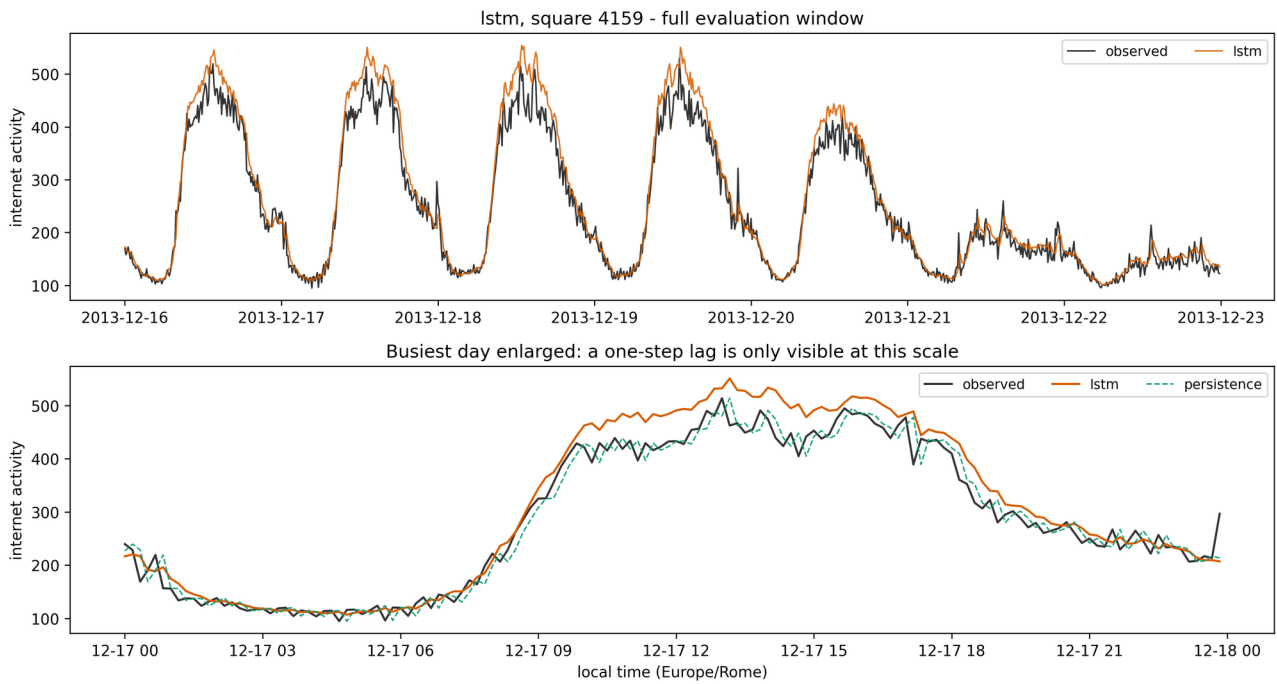


Figure 15 — LSTM, square 4159. The morning ramp is where this model fails; see Section 7.2.

Square 4556 (rank 109, highest night floor).

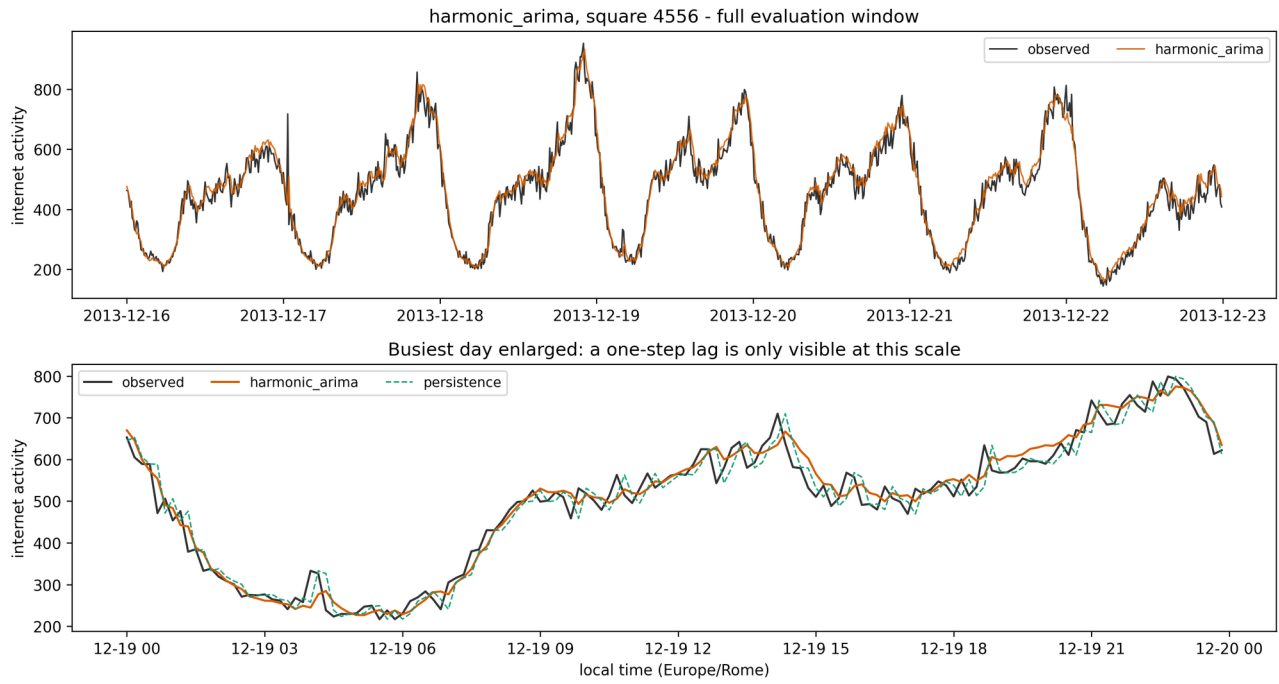


Figure 16 — Harmonic ARIMA, square 4556.

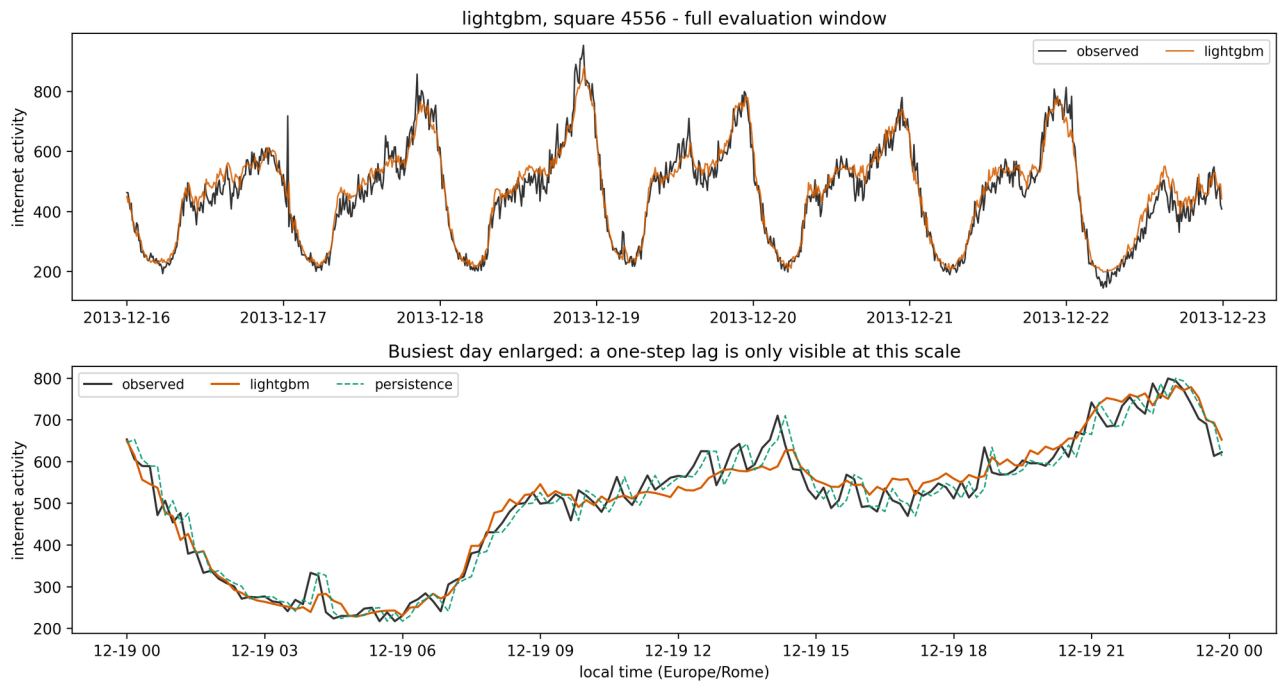


Figure 17 — LightGBM, square 4556.

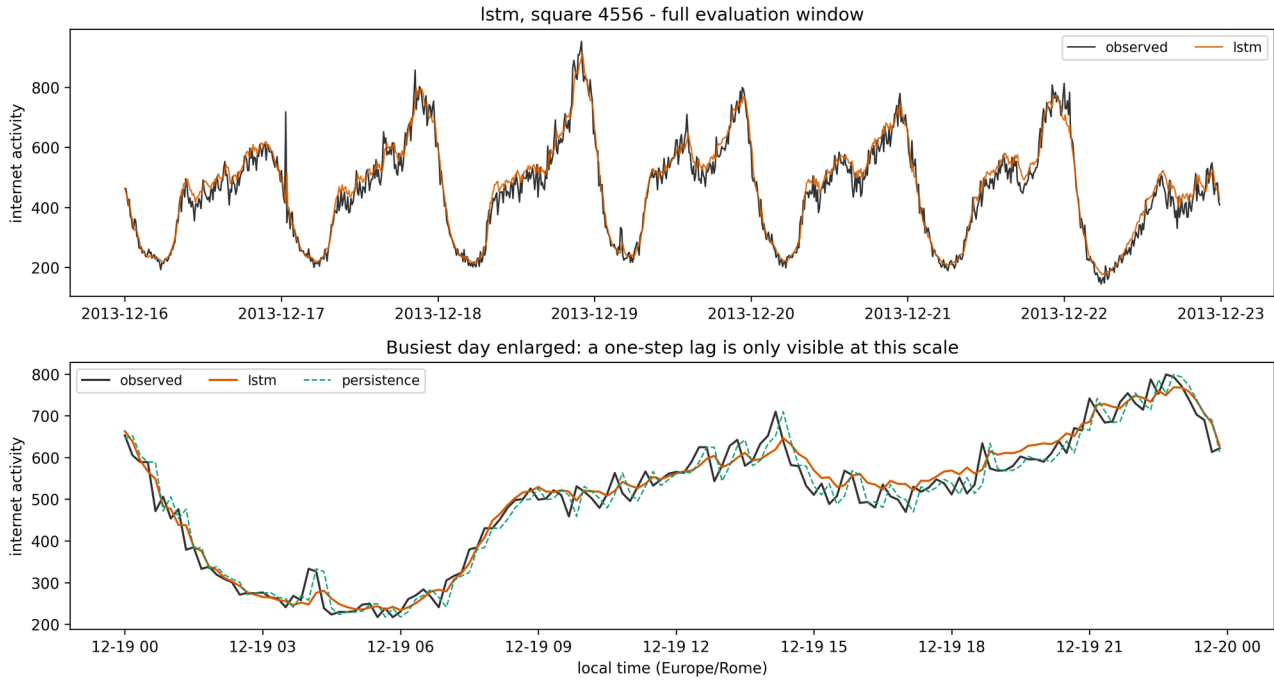


Figure 18 — LSTM, square 4556.

6.3 Seed variance

The three-seed protocol is load-bearing rather than ceremonial. On square 4159 the LSTM scores 21.16 ± 4.59 MAE, a relative standard deviation of 22%. On square 4556 it scores 28.85 ± 1.79 against persistence's 28.86 — a difference far smaller than the spread, so the two are not distinguishable on this evidence. A single-seed result would have been a number with no interpretation attached.

6.4 Computational cost

Table 12 — Computational cost, square 5161, 1,008 one-step forecasts. Training times are **not comparable across devices**; the device is carried in the table rather than left to a caption.

Model	Device	Train	Inference	Per step	Parameters
harmonic ARIMA	CPU	18.0 s	75.2 s	74.6 ms	22
LightGBM	CPU	5.0 s	0.022 s	0.022 ms	19,639
LSTM	GPU (T4)	9.9 s	0.041 s	0.041 ms	202,369
LSTM ensemble	GPU (T4)	29.7 s	0.124 s	0.123 ms	607,107
persistence / seasonal naive	CPU	0.0 s	< 0.001 s	< 0.001 ms	0

Cost inverts between training and deployment. The most accurate and by far the smallest model is also the most expensive to serve: harmonic ARIMA is roughly **1,825× slower per forecast** than the LSTM, because appending an observation without refitting still runs a Kalman filter update at every step.

For a single area this does not disqualify it — 75 seconds to produce a week of forecasts is comfortably inside a ten-minute budget. It does disqualify it at city scale. Producing one forecast for each of the 10,000 grid cells would take **12.4 minutes** at 74.6 ms per step, which is *longer than the ten-minute interval being forecast*, so the model could not keep pace with the data in real time. The LSTM would need 0.41 seconds for the same pass. The accuracy ranking and the deployability ranking are therefore opposites.

The LSTM's training cost depends on hardware to a degree that changes what is feasible. For one configuration fitted on both machines — window 288, 64 hidden units, 2 layers — the same fit took **2,019 seconds on the CPU and 2.6 seconds on the T4**, a measured **777×** difference. The largest configuration reached during the CPU sweep took **13.8 hours** for a single fit; its GPU counterpart was never run, so that figure is a CPU cost rather than half of a ratio. The sweep as a whole was abandoned locally after 16 hours with two of five stages complete, and finished on the GPU in 2.9 minutes. A 288-step recurrence is latency-bound on a sequential dependency that CPU threads cannot divide: the process held an average of 0.88 of 8 available cores throughout, so more cores would not have helped.

7. Discussion

7.1 Did the models learn anything, or copy the last value?

With a lag-1 autocorrelation of 0.987 a model can achieve a respectable error by learning to repeat its most recent input, and would appear successful while having learned nothing. This was tested explicitly.

The measure used is the **copy ratio**: the mean distance between a model's forecasts and the persistence baseline, divided by how far the series itself moves between steps. Zero means the two are the same forecast.

Table 13 — Collapse-to-persistence check, test week.

Model	copy ratio (range across areas)	verdict
persistence	0.00	collapsed by definition
harmonic ARIMA	0.54 – 0.64	independent
LSTM	0.71 – 1.14	independent
LightGBM	0.77 – 0.93	independent
seasonal naive	2.70 – 3.68	independent

No model collapsed to persistence. The closest is harmonic ARIMA on square 4159 at 0.54, which is also its best result — the model is genuinely close to persistence there, and beats it.

One methodological point is worth stating, because the obvious test gives the wrong answer.

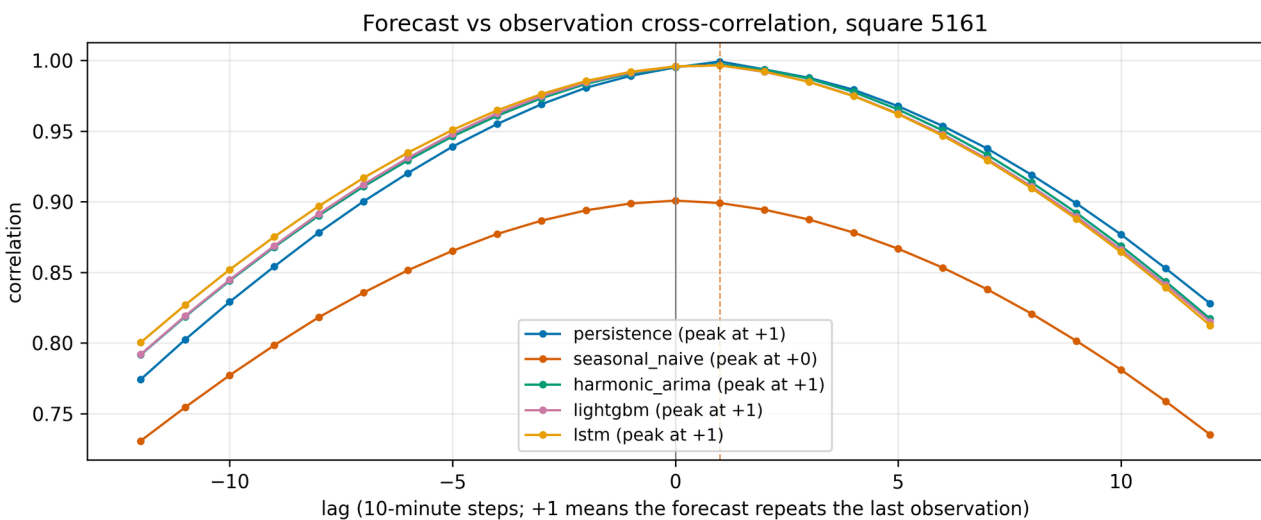


Figure 19 — Forecast-to-observation cross-correlation, square 5161. Every model except seasonal naive peaks at lag +1.

Cross-correlating forecasts against observations, every model — including the best — peaks at **lag +1**. This is *not* evidence of copying. A one-step forecast is constructed only from observations up to $t-1$, so it cannot contain the innovation at t , and must correlate slightly more strongly with the previous observation than with the current one. A forecast peaking at lag 0 on a series this persistent would be the suspicious case, because it would imply access to the present value. The measured data settles the question: seasonal naive is the only model peaking at lag 0, and it is comfortably the worst forecaster in the study. The verdict in Table 13 therefore rests on the copy ratio, with the peak reported but not used as the test.

7.2 Where the errors fall

Absolute error scales with traffic level, so every model's error follows the diurnal cycle. What distinguishes the models is where their error peaks *relative* to the others.

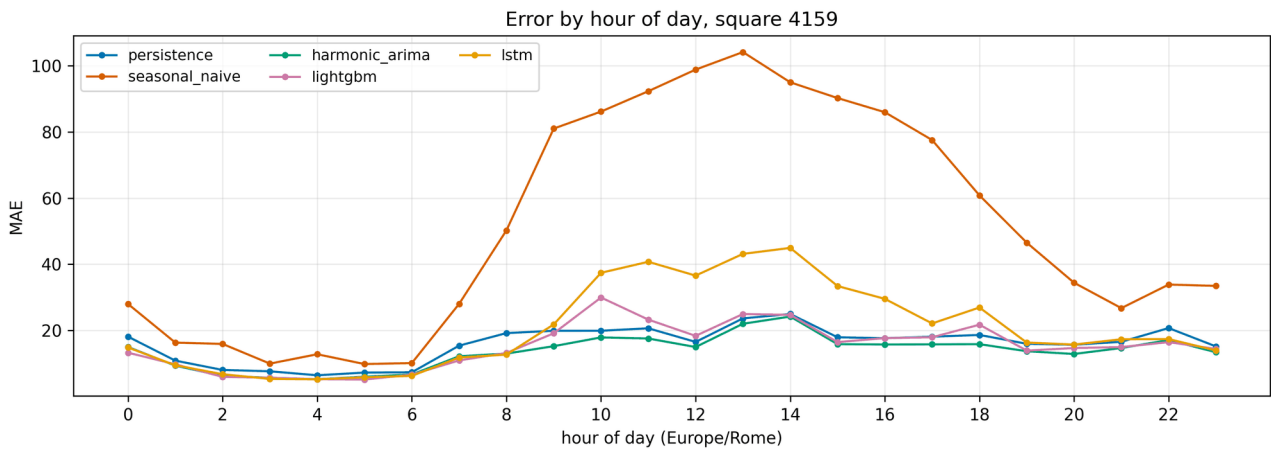


Figure 20 — Mean absolute error against hour of day, square 4159, one line per model. The LSTM's excess is concentrated in the morning.

Table 14 — Error by day type, test week.

Area	Model	Weekday MAE	Weekend MAE	Weekend penalty
5161	persistence	88.36	103.90	1.18
5161	harmonic ARIMA	74.96	105.33	1.41
5161	LightGBM	82.65	96.45	1.17
5161	LSTM	77.26	89.62	1.16
4159	persistence	17.41	12.31	0.71
4159	harmonic ARIMA	14.32	11.87	0.83
4159	LightGBM	16.20	12.74	0.79
4159	LSTM	24.13	12.03	0.50
4556	persistence	29.53	27.19	0.92
4556	harmonic ARIMA	25.83	25.97	1.01
4556	LightGBM	29.14	31.91	1.10
4556	LSTM	27.54	29.01	1.05

Table 15 — Worst contiguous six-hour windows, test week. ratio above 1 means persistence would have been better over exactly that stretch.

Area	Model	Window start	MAE	persistence MAE	ratio
4159	LSTM	2013-12-17 10:00	52.2	20.5	2.55
4159	LSTM	2013-12-18 09:50	60.2	26.4	2.28
4159	LSTM	2013-12-19 09:50	53.0	27.0	1.96
4556	LightGBM	2013-12-22 11:50	52.2	30.0	1.74
4159	LightGBM	2013-12-16 10:00	33.1	19.3	1.72
4556	LightGBM	2013-12-16 11:40	44.6	28.2	1.58

The LSTM's poor result on square 4159 is a weekday-morning failure, not a general one. Its three worst six-hour windows all begin at approximately 09:50 on consecutive weekdays (17, 18 and 19 December), at 2.0 to 2.6 times the persistence error over the same stretches. Its weekday MAE of 24.13 against a weekend MAE of 12.03 is the most lopsided split of any model — persistence, by comparison, sits at 17.41 and 12.31. The model misses the morning ramp on the area with the least traffic, and is competitive there at weekends.

Harmonic ARIMA's advantage on square 5161 is weekday-specific in the opposite direction. It is the best model of any on weekdays (74.96 MAE) but worse than persistence at weekends (105.33 against 103.90), a weekend penalty of 1.41. Its overall win on that area is earned entirely on working days.

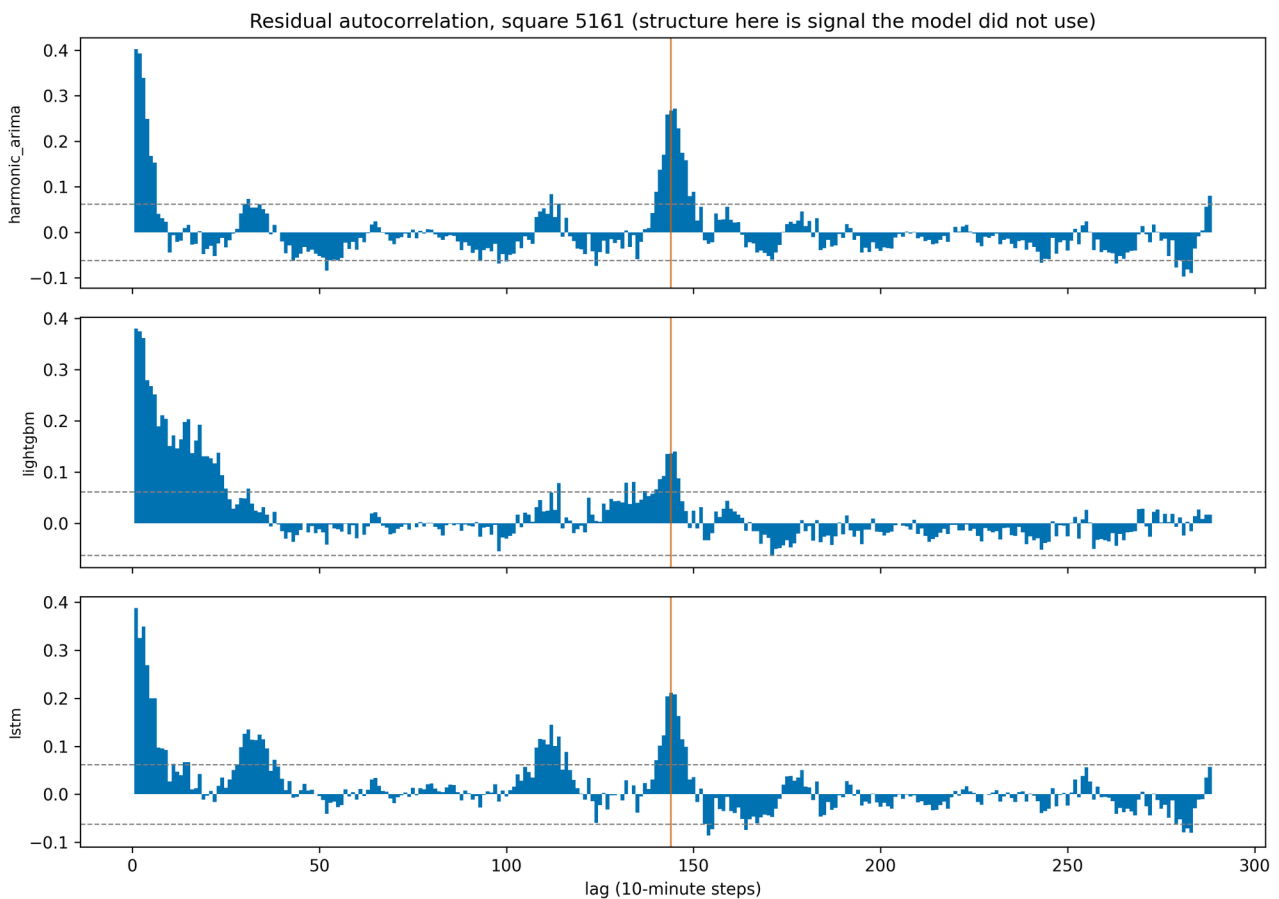


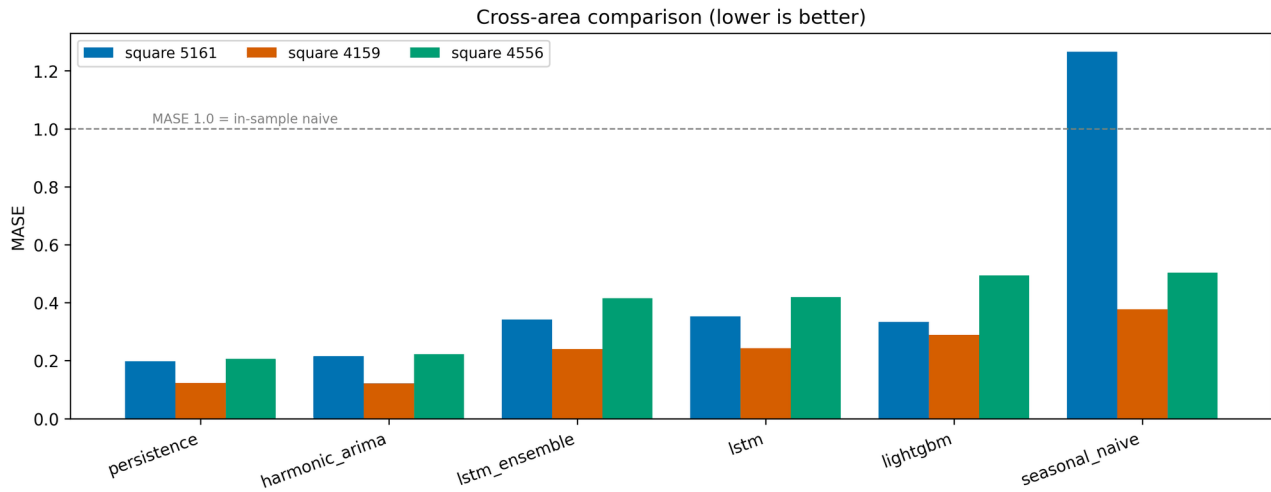
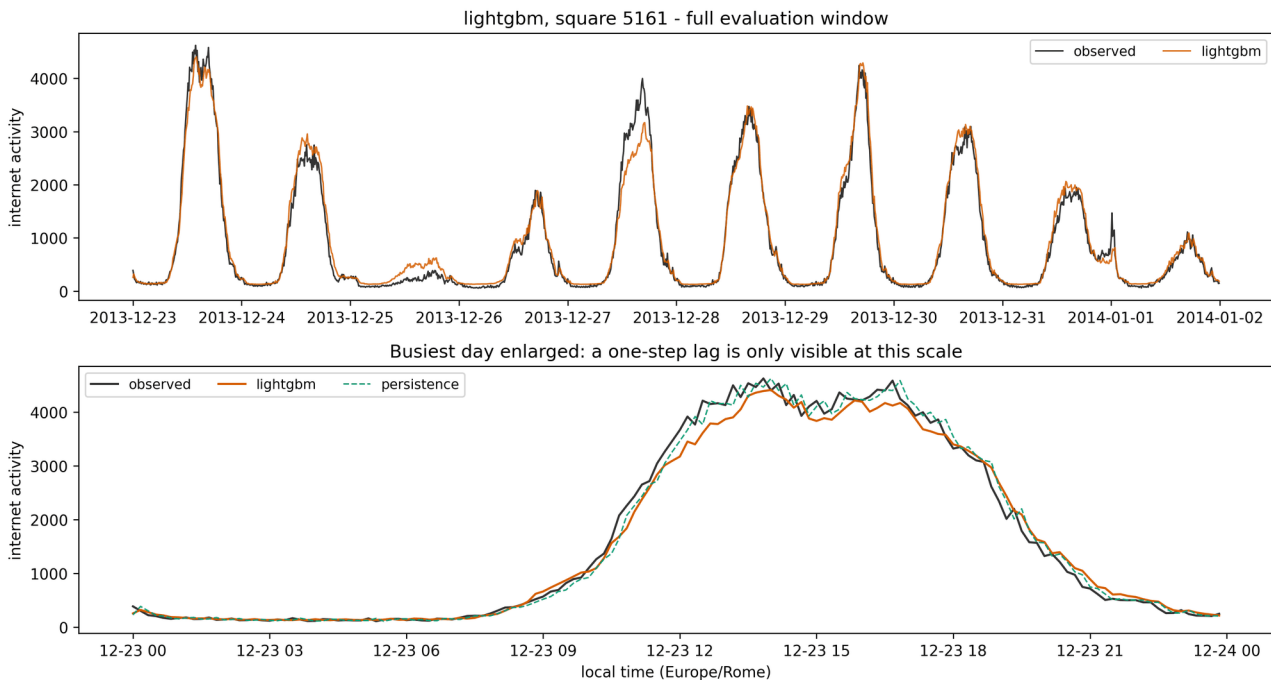
Figure 21 — Residual autocorrelation per model, square 5161. Structure remaining here is signal the model did not use; the marked line is the daily period.

7.3 The holiday stress split

The stress split covers 23 December to 1 January, contains four of the eight Italian public holidays in the study period, and was never tuned on or used for selection.

Table 16 — MASE on the held-out stress split, relative to persistence on the same area.

Model	5161	4159	4556
persistence (absolute MASE)	0.198	0.124	0.206
harmonic ARIMA	+9.0%	−1.9%	+7.9%
LightGBM	+68.6%	+133.1%	+139.8%
LSTM	+78.0%	+95.8%	+103.2%
seasonal naive	+538.6%	+204.7%	+144.7%

**Figure 22 — MASE per model on the stress split.****Figure 23 — LightGBM on square 5161 over the holiday period, where it degrades by 68.6% relative to persistence.**

Persistence wins outright on two of the three areas. Both learned models degrade severely — LightGBM by up to 140% and the LSTM by up to 103% — while the harmonic model stays within 9% and beats persistence on one area.

The two baselines also move in **opposite directions**, which isolates what the holidays do to the series. Persistence gets easier (MASE $0.267 \rightarrow 0.198$ on square 5161) because the traffic becomes smoother, while seasonal naive collapses past MASE 1.0 ($0.975 \rightarrow 1.266$) — worse than the in-sample naive forecast it is normalised against — because the weekly pattern it depends on is precisely what Christmas and New Year destroy.

LightGBM's failure was **predicted before the split was run**. The model's design documentation states that a tree ensemble cannot extrapolate beyond the range of its training targets, and identifies the stress split as where that limitation should bite. It is the worst non-seasonal-naive model on all three areas, with R^2 falling to 0.474 on square 4159 against persistence's 0.878. This is the clearest result in the study: the ranking obtained on a well-behaved week does not survive a distribution shift, and the model with the most capacity to fit the training distribution is the one that fails hardest outside it.

7.4 Limitations

- **Hyperparameters were tuned on one area only.** The LSTM's failure on square 4159 is partly a *transfer* result: it inherited a configuration and epoch count selected on the highest-traffic area. Per-area tuning was outside the compute budget, so the evidence does not separate architectural limitation from transfer failure.
- **One test week.** The reported results rest on a single seven-day window fixed by the brief. The stress split provides a second, deliberately harder window, but neither is a substitute for repeated evaluation across many weeks.
- **The staged search is not device-invariant.** Run on CPU the LSTM sweep selected a sequence length of 288; run on the GPU it selected 144, because a candidate that stopped at epoch 17 on one device stopped at epoch 2 on the other. Different kernels give different numerics, and early stopping with patience 5 on a noisy validation curve turns that into a different architectural choice. The selection should not be presented as inevitable.
- **Three areas.** Conclusions about how performance varies with area characteristics rest on three points.
- **Univariate.** No cross-cell or exogenous information (weather, events) is used, though the anomaly analysis showed holidays carry a measurable effect and the literature [3] shows spatial information helps.
- **ARIMA order selected on a margin within noise.** ARIMA(3,0,1) was chosen over ARIMA(1,0,1) on a validation MAE difference of 0.08, which two extra parameters do not justify on the evidence; the protocol's selection rule was followed rather than overridden after the fact.

8. Conclusion and Future Work

Three sequential models were compared for one-step-ahead forecasting of mobile internet traffic across three areas of Milan, against persistence and seasonal-naive baselines.

The **dynamic harmonic regression performs best**, with the lowest mean MASE (0.213) and the best result on two of three test-week areas, improving on persistence by 11.3%. It does so with 22 parameters, against LightGBM's 19,639–21,010 and the LSTM's 202,369. On the remaining area a three-seed LSTM ensemble is better, so no single model dominates — which is itself the answer to the second half of the research question: the ranking depends on the area, and the areas were chosen to differ.

Three findings are worth carrying forward.

Complexity bought accuracy only while the distribution held still. On the held-out holiday period persistence wins two of three areas outright, and both learned models degrade by 69–140% while the smallest model stays within 9%. The ranking obtained on the test week does not survive a distribution shift.

The baseline is the result. With lag-1 autocorrelation at 0.987, persistence achieves MASE 0.195–0.267 and R^2 above 0.94, and only two of three models beat it on average. A study reporting only model errors, without that floor, would present a respectable-looking result that is worse than doing nothing. This supports the qualification Azari *et al.* [2] attach to their own headline finding — that there are conditions under which the simpler statistical model is close to optimal at far lower complexity — rather than the headline itself.

Cost inverts between training and deployment. The most accurate and smallest model is roughly $1,825\times$ the most expensive to serve, and at city scale could not keep pace with the interval it forecasts. The LSTM, meanwhile, is only trainable in reasonable time on a GPU: an identical fit took 2,019 seconds on a CPU against 2.6 seconds on a T4. Reporting a single "training time" column without the device attached would have been meaningless.

Future work

- **Per-area tuning**, to separate the LSTM's architectural limits from transfer failure — the single most informative extension, because it addresses the study's main limitation directly.
- **Multi-step horizons**. Every conclusion here concerns a ten-minute horizon, where persistence is strong; the ranking would likely change at one hour or one day, and the relative standing of the models is a function of the horizon rather than a property of the methods.
- **Holiday-aware features or a regime-switching model**, since the failure on the stress split is specific and predictable rather than random.
- **Cross-cell information**. Neighbouring cells are highly correlated and Zhang and Patras [3] demonstrate that spatial structure helps materially; a graph or convolutional model could use what the univariate setting discards.

References

- [1] G. Barlacchi, M. De Nadai, R. Larcher, A. Casella, C. Chitic, G. Torrisi, F. Antonelli, A. Vespignani, A. Pentland, and B. Lepri, "A multi-source dataset of urban life in the city of Milan and the Province of Trentino," *Scientific Data*, vol. 2, art. 150055, 2015. doi: 10.1038/sdata.2015.55
- [2] A. Azari, P. Papapetrou, S. Denic, and G. Peters, "Cellular traffic prediction and classification: A comparative evaluation of LSTM and ARIMA," in *Discovery Science (DS 2019)*, Lecture Notes in Computer Science, Springer, 2019, pp. 129–144. doi: 10.1007/978-3-030-33778-0_11
- [3] C. Zhang and P. Patras, "Long-term mobile traffic forecasting using deep spatio-temporal neural networks," in *Proc. 19th ACM Int. Symp. Mobile Ad Hoc Networking and Computing (MobiHoc '18)*, Los Angeles, CA, USA, 2018, pp. 231–240. doi: 10.1145/3209582.3209606
- [4] G. L. Santos *et al.*, "Predicting short-term mobile Internet traffic from Internet activity using recurrent neural networks," *International Journal of Network Management*, vol. 32, no. 3, art. e2191, 2022. doi: 10.1002/nem.2191
- [5] X. Wang, Z. Wang, K. Yang, Z. Song, C. Bian, J. Feng, and C. Deng, "A survey on deep learning for cellular traffic prediction," *Intelligent Computing*, vol. 3, art. 0054, 2024. doi: 10.34133/icomputing.0054

- [6] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>
- [7] K. Bandara, R. J. Hyndman, and C. Bergmeir, "MSTL: A seasonal-trend decomposition algorithm for time series with multiple seasonal patterns," *International Journal of Operational Research*, vol. 52, no. 1, 2025.
- [8] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M5 competition: Conclusions," *International Journal of Forecasting*, 2022. doi: 10.1016/j.ijforecast.2022.04.006
- [9] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, Long Beach, CA, USA, 2017, pp. 3146–3154.
- [10] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, 2006. doi: 10.1016/j.ijforecast.2006.03.001
- [11] Telecom Italia, "Telecommunications — SMS, Call, Internet — MI," Harvard Dataverse, 2015. doi: 10.7910/DVN/EGZHFV
- [12] Telecom Italia, "Milano Grid," Harvard Dataverse, 2015. doi: 10.7910/DVN/QJWLFU
- [13] Polars contributors, "Polars: DataFrames for the new era." [Online]. Available: <https://pola.rs/>
- [14] S. Seabold and J. Perktold, "statsmodels: Econometric and statistical modeling with Python," in *Proc. 9th Python in Science Conf. (SciPy 2010)*, 2010.
- [15] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019, pp. 8024–8035.

Appendix A — Reproducibility

The analysis is reproducible from a clean checkout without the 19.38 GiB download. Four artefacts are version-controlled for this purpose: the extracted series (283 KB), the study-area selection, the tuned hyperparameters, and the prediction series (312 KB).

```
git clone <repository URL> && cd milan-traffic-forecasting
python -m venv .venv && .venv/Scripts/activate # source .venv/bin/activate on Linux
pip install -r requirements.txt

python run.py test # 501 tests
python run.py evaluate --split test # baseline metrics from the committed series
python run.py results --split test # figures and diagnostic tables
python run.py results --split stress
```

Verification. A fresh clone into an empty virtual environment was confirmed to reproduce every metric and every summary in this report **exactly**, not merely to three significant figures. The only differences between regenerated and committed artefacts are wall-clock timing columns and line endings.

Three stages cannot run from the committed artefacts, for stated reasons: ingest requires the 19.38 GiB of raw text, which is not redistributable; part of the exploratory analysis requires the 341 MB full matrix, which is too large to commit; and the LSTM final fits require a CUDA device, since on a CPU the nine fits take approximately 31 hours. The prediction series are committed precisely because of that last constraint.

Determinism is enforced by fixed seeds for Python, NumPy and PyTorch, with library versions and hardware recorded alongside the results. Harmonic ARIMA and LightGBM reproduced bit-identically across two different machines; the LSTM did not, for the reason given in Section 7.4.

Appendix B — Supplementary figures

The following figures are produced by the analysis and referenced collectively rather than individually in the body of this report. All are at 300 dpi in report/figures/.

Figure family	Files	Shows
Error-by-hour	error_by_hour_test_{5161,4556}.png	Error against hour of day for the two areas not shown as Figure 20
Error heatmaps	error_heatmap_test_{area}_{model}.png (9)	Error over day-of-week $\times$ hour-of-day per model per area
Residual ACF	residual_acf_test_{4159,4556}.png	Residual autocorrelation for the two areas not shown as Figure 21
Cross-correlation	cross_correlation_test_{4159,4556}.png	Copying diagnostic for the two areas not shown as Figure 19
Stress forecasts	forecast_stress_{area}_{model}.png (9)	All three models on all three areas over the holiday period

Appendix C — Declaration on the use of AI tools

AI was used to help interpret and clarify the project requirements, which supported effective planning and enabled me to break the project into manageable phases. The implementation, development decisions, and final work were completed and reviewed by me.

Source code: <https://github.com/Hassan-Adelani-Luqman/milan-traffic-forecasting> **Demonstration video:** <video URL>